



Курс «Глубокое обучение»

Нейронные сети

Александр Дьяконов

План

Простейшая нейросеть – 1 нейрон

Функции активации

(линейная, пороговая, сигмоида, гиперболический тангенс, softmax, LeakyReLU, ELU, Maxout, Exponential Linear Unit, Maxout, Gaussian Error Linear Unit)

Функциональная выразимость нейрона

Теорема об универсальной аппроксимации

Сеть прямого распространения

Необходимость глубоких сетей

Обучение

Обратное распространение градиента (Backpropagation)

Функции ошибки

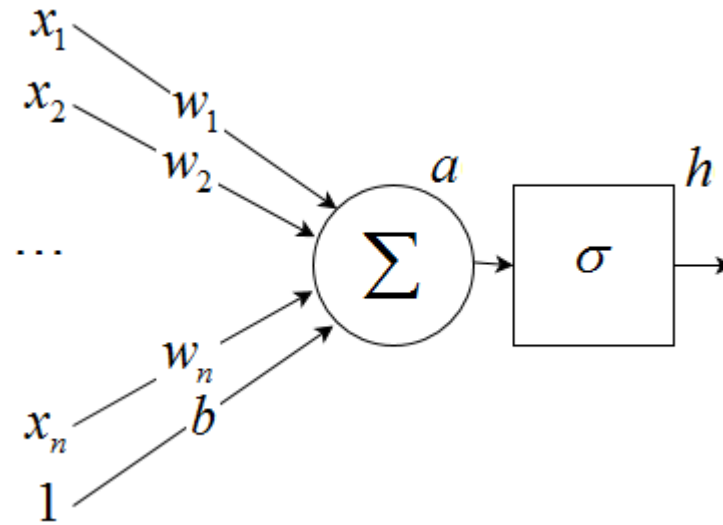
Нейросеть – вычислительный граф

Вычисление градиента на графе

Производные на компьютере

Проблема затухания градиента

Простейшая нейросеть – 1 нейрон



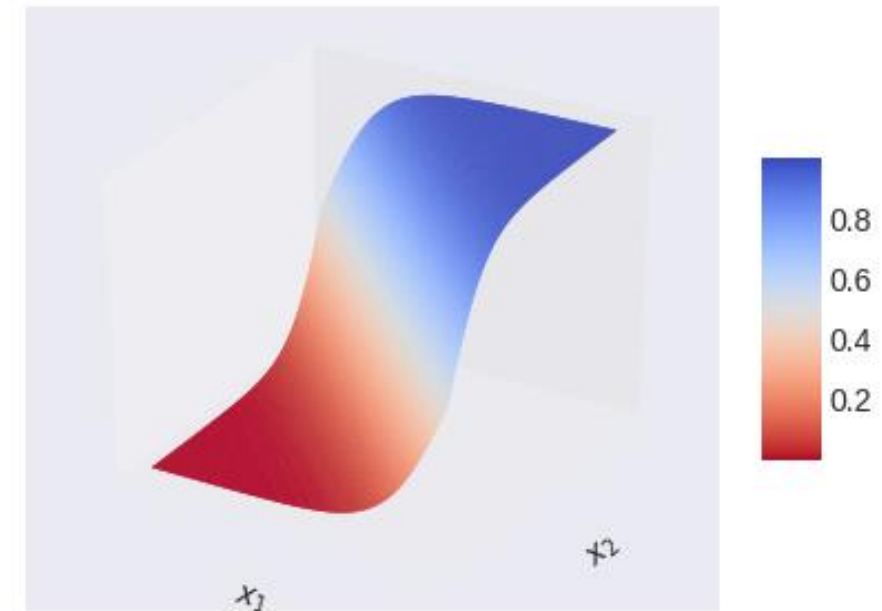
$$a(x) = b + w_1x_1 + \dots + w_nx_n = \sum_{t=0}^n w_t x_t$$

$$h(x) = \sigma(a(x))$$

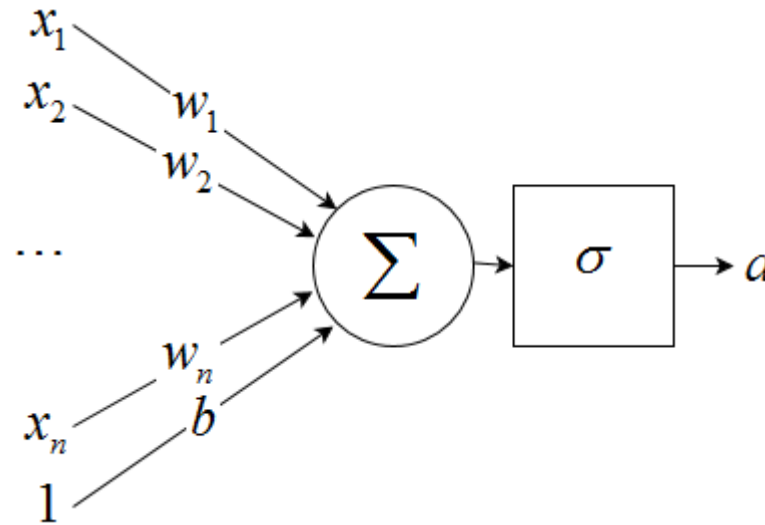
b – смещение

w_t – веса связей

σ – функция активации



Линейные модели – нейросети!



Линейная регрессия

$$a(x) = b + w_1x_1 + \dots + w_nx_n$$

Линейный классификатор

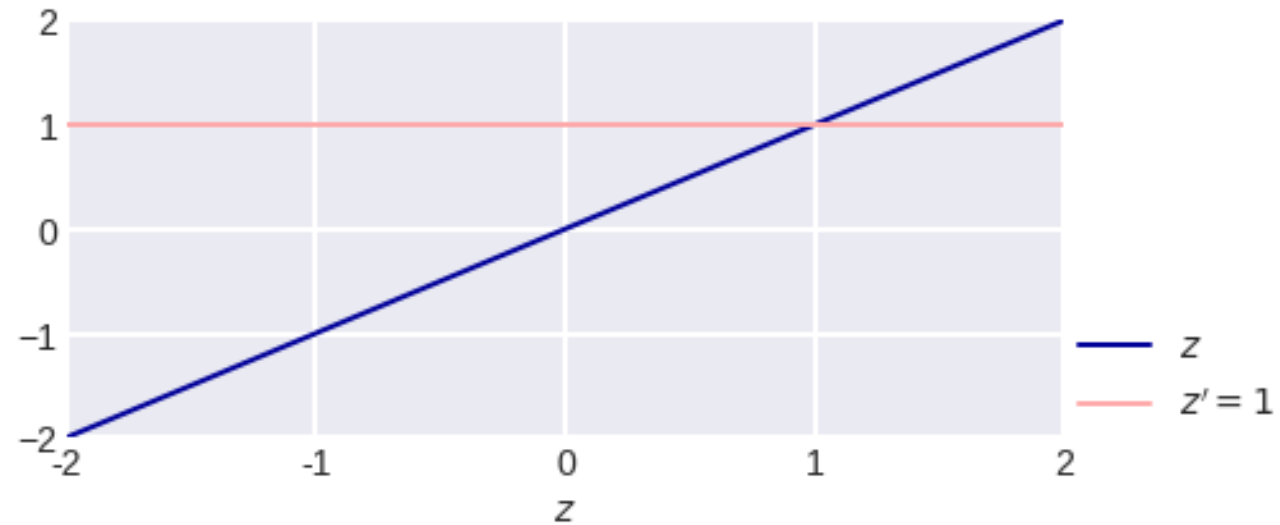
$$a(x) = \text{th}(b + w_1x_1 + \dots + w_nx_n)$$

Логистическая регрессия

$$a(x) = \sigma(b + w_1x_1 + \dots + w_nx_n)$$

Функции активации

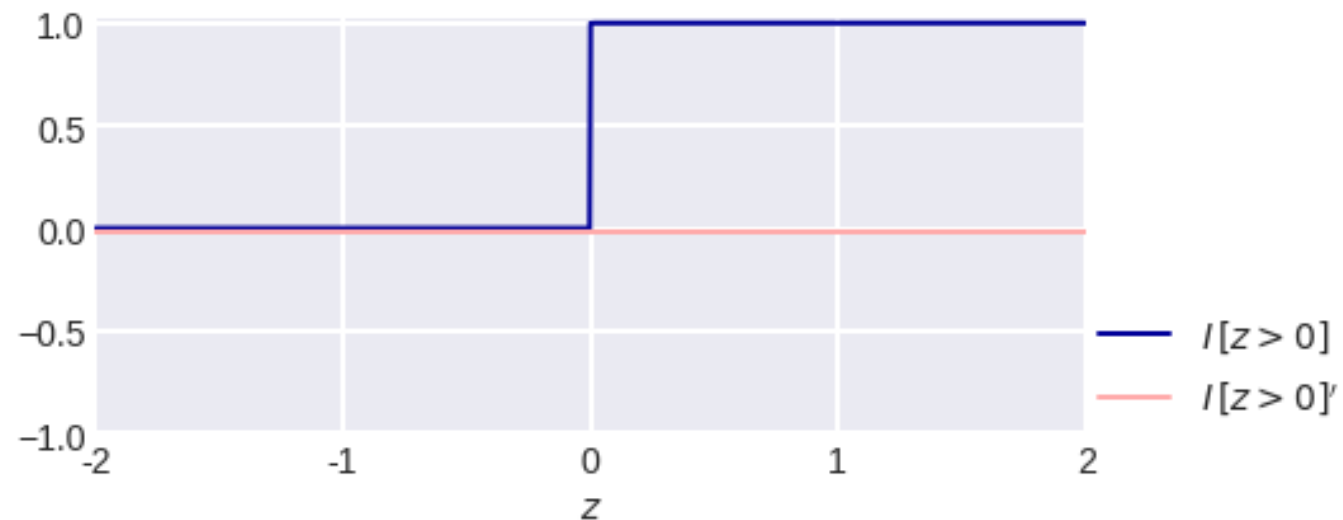
Тождественная функция (линейная / linear activation function)



$$f(z) = z$$

$$\frac{\partial f(z)}{\partial z} = 1$$

Пороговая функция (threshold function)

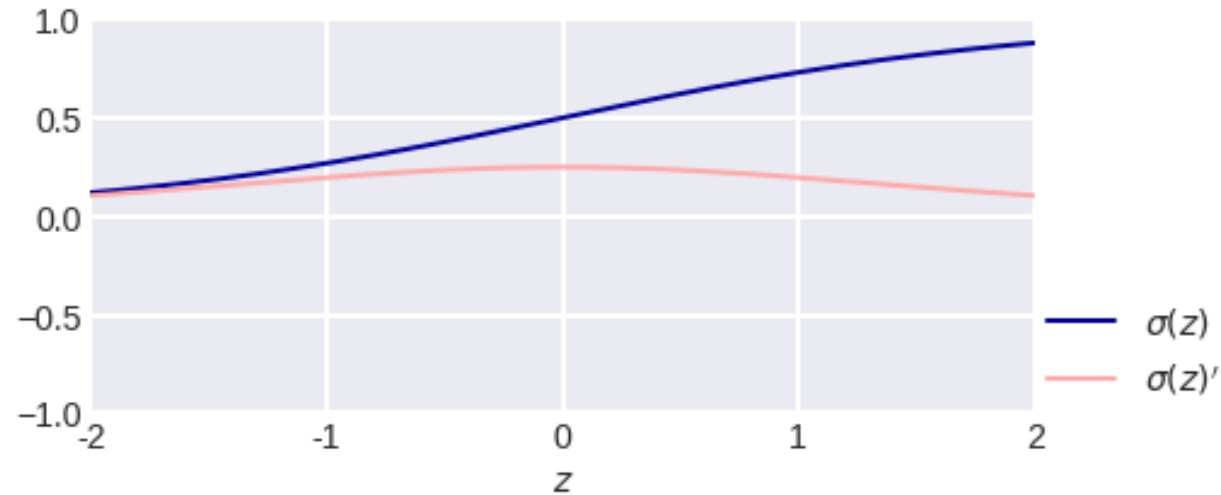


$$\text{th}(z) = I[z > 0]$$

$$\frac{\partial \text{th}(z)}{\partial z} = 0$$

Функции активации

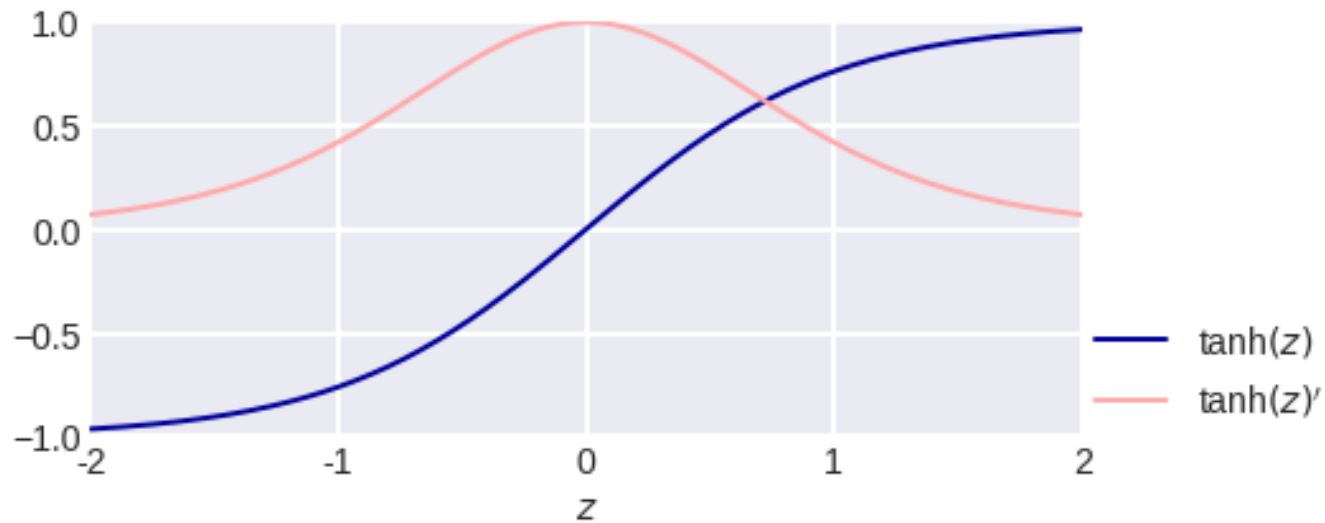
Сигмоида (sigmoid activation function)



$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

$$\frac{\partial \sigma(z)}{\partial z} = \sigma(z)(1 - \sigma(z)) > 0$$

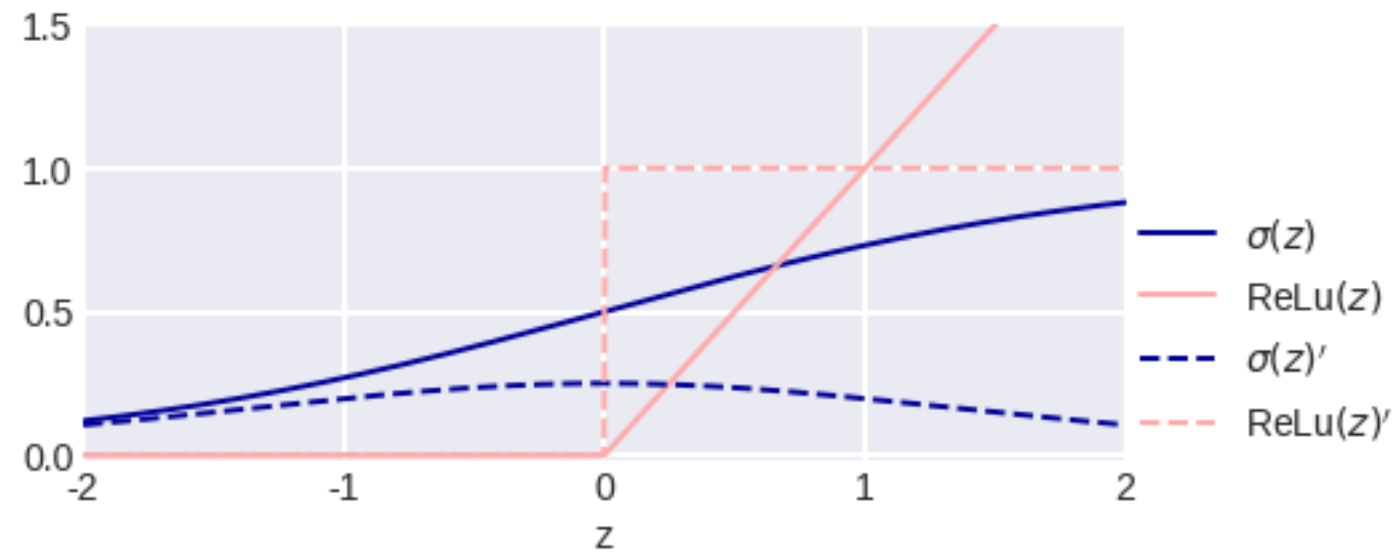
Гиперболический тангенс (hyperbolic tangent)



$$\tanh(z) = \frac{2}{1 + e^{-2z}} - 1 = \frac{e^{+z} - e^{-z}}{e^{+z} + e^{-z}} = \frac{e^{+2z} - 1}{e^{+2z} + 1}$$

$$\frac{\partial \tanh(z)}{\partial z} = 1 - \tanh^2(z)$$

Функции активации



ReLU = Rectified Linear Unit

$$\text{ReLU}(z) = \max(0, z)$$

$$\frac{\partial \text{ReLU}(z)}{\partial z} = I[z > 0]$$

Эту функцию будем использовать в примерах...
потом разберёмся, чем она хороша

Функции активации

обобщение сигмoиды в задачах классификации

$$\text{softmax}(z_1, \dots, z_k) = \frac{1}{\sum_{t=1}^k \exp(z_t)} (\exp(z_1), \dots, \exp(z_k))^T$$

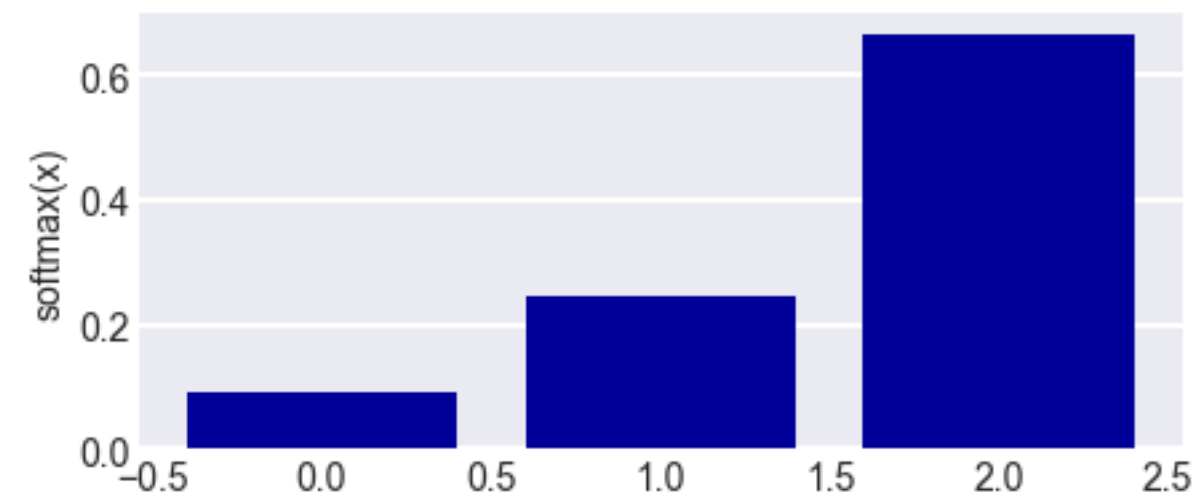
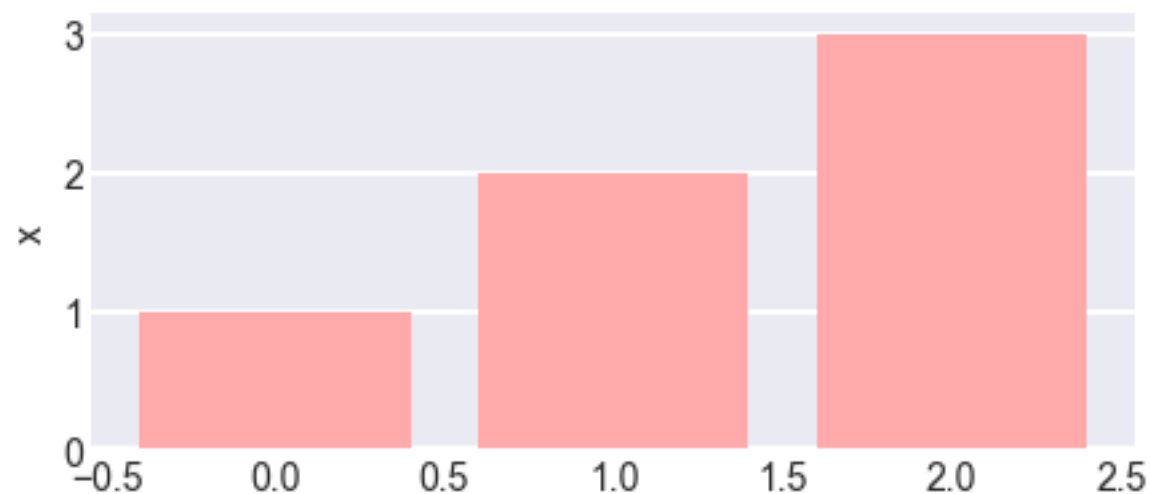
выходы положительные, сумма выходов = 1
выходы интерпретируются как вероятности

$$[0.5, 0.5, 0.1, 0.7] \rightarrow [0.257, 0.257, 0.172, \mathbf{0.314}]$$

$$[-1.0, 0, 1.0, 0, -1.0] \rightarrow [0.07, 0.18, \mathbf{0.5}, 0.18, 0.07]$$

$$[1.0, 1.0, 1.0, 2.0, 1.0] \rightarrow [0.15, 0.15, 0.15, \mathbf{0.4}, 0.15]$$

Минутка кода



```
from torch.nn.functional import softmax
x = torch.tensor([1, 2, 3], dtype=float)
y = softmax(x, dim=0) # указать размерность
```

Не всегда softmax нужен в явном виде (например, при NLLLoss)

Минутка кода

```
from torch.nn import Sigmoid
from torch.nn.functional import sigmoid
x = torch.tensor([1, 2, 3], dtype=float)
s = Sigmoid() # создаёт nn.Module
s(x), sigmoid(x) # один результат
```

Все нужные функции активации есть...
<https://pytorch.org/docs/1.9.0/nn.functional.html>

Non-linear activation functions

`threshold`

Thresholds each element of the input Tensor.

`threshold_`

In-place version of `threshold()`.

`relu`

Applies the rectified linear unit function element-wise.

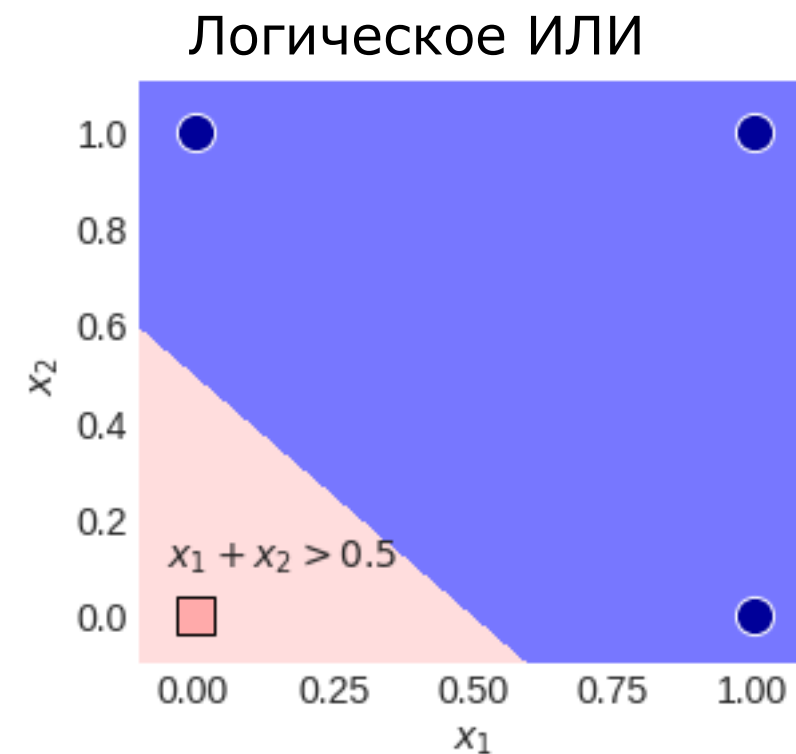
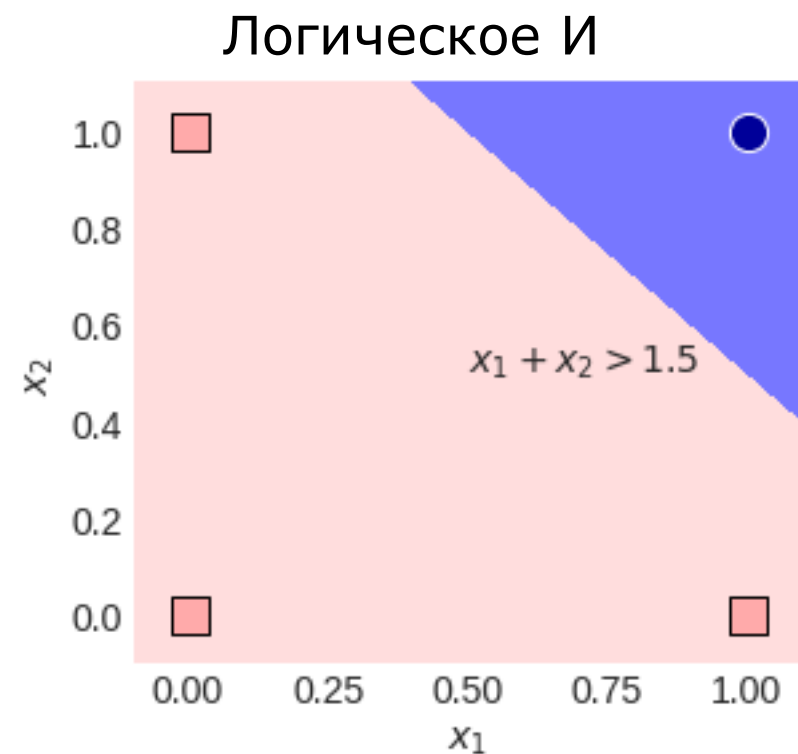
`relu_`

In-place version of `relu()`.

`hardtanh`

Applies the HardTanh function element-wise.

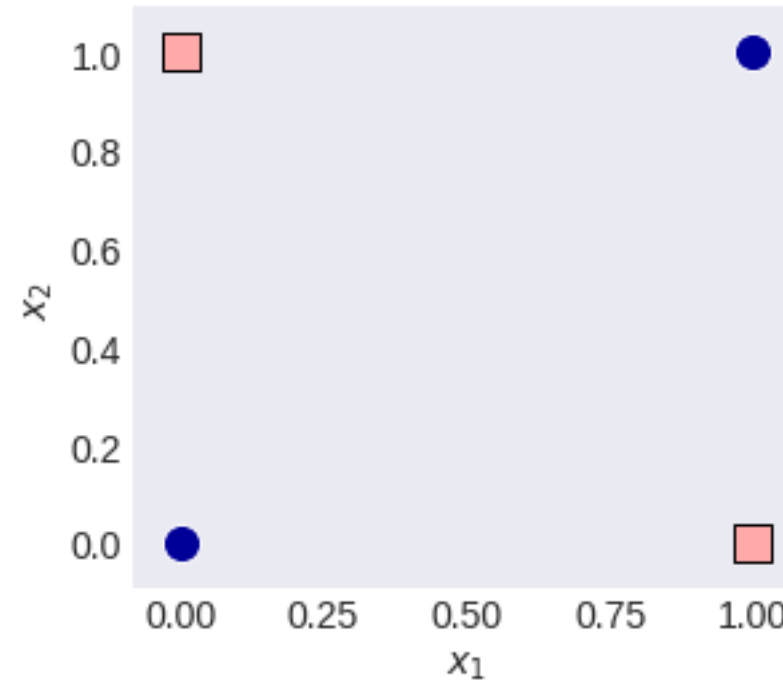
Что может один нейрон



Для простоты – пороговая функция активации

Что НЕ может один нейрон

Исключающее ИЛИ / эквивалентность



$$\text{th}(\text{th}(x_1 + x_2 - 1.5) + \text{th}(-x_1 - x_2 + 0.5) - 0.5)$$

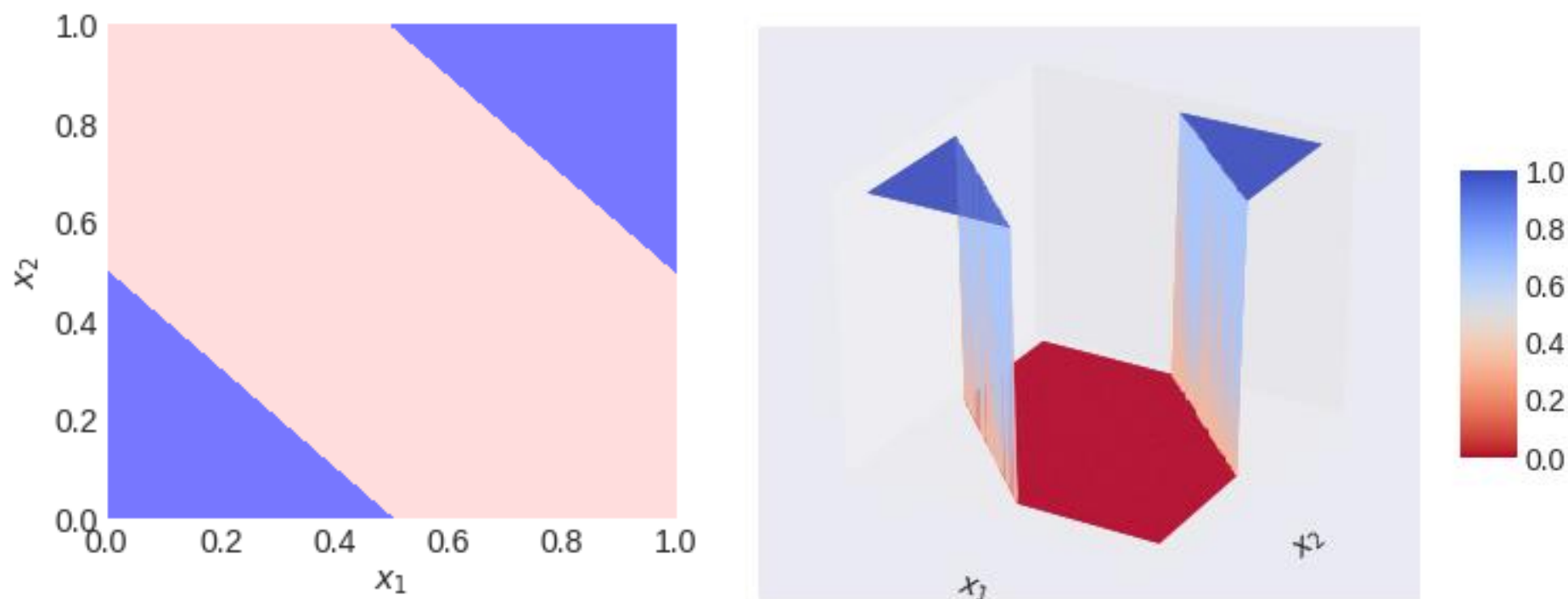
$$\text{th}(\text{th}(0 + 0 - 1.5) + \text{th}(-0 - 0 + 0.5) - 0.5) = \text{th}(0 + 1 - 0.5) = 1$$

$$\text{th}(\text{th}(0 + 1 - 1.5) + \text{th}(-0 - 1 + 0.5) - 0.5) = \text{th}(0 + 0 - 0.5) = 0$$

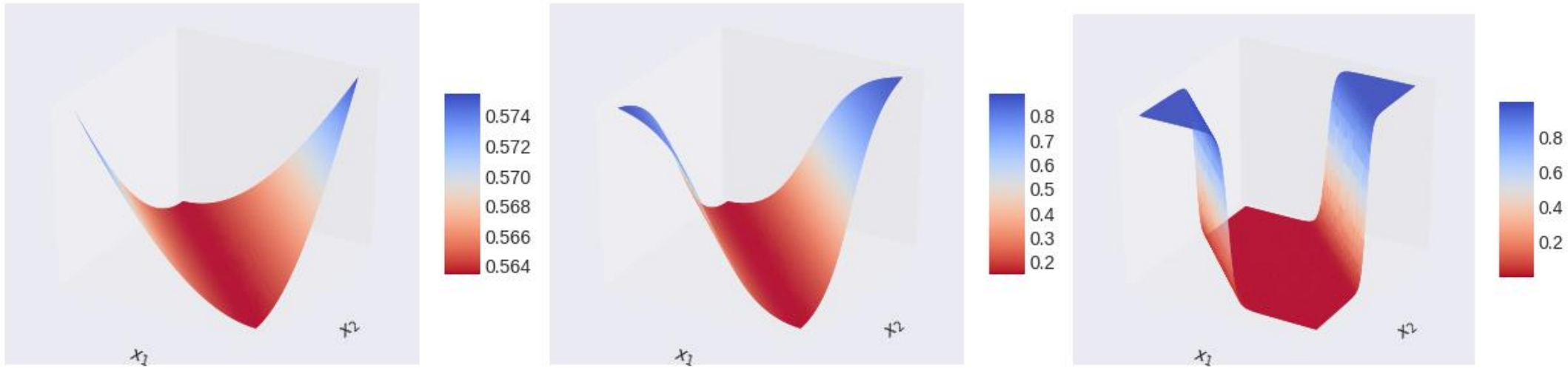
$$\text{th}(\text{th}(1 + 0 - 1.5) + \text{th}(-1 - 0 + 0.5) - 0.5) = \text{th}(0 - 0 - 0.5) = 0$$

$$\text{th}(\text{th}(1 + 1 - 1.5) + \text{th}(-1 - 1 + 0.5) - 0.5) = \text{th}(1 + 0 - 0.5) = 1$$

Что НЕ может один нейрон



Сигмоида стремится к пороговой функции



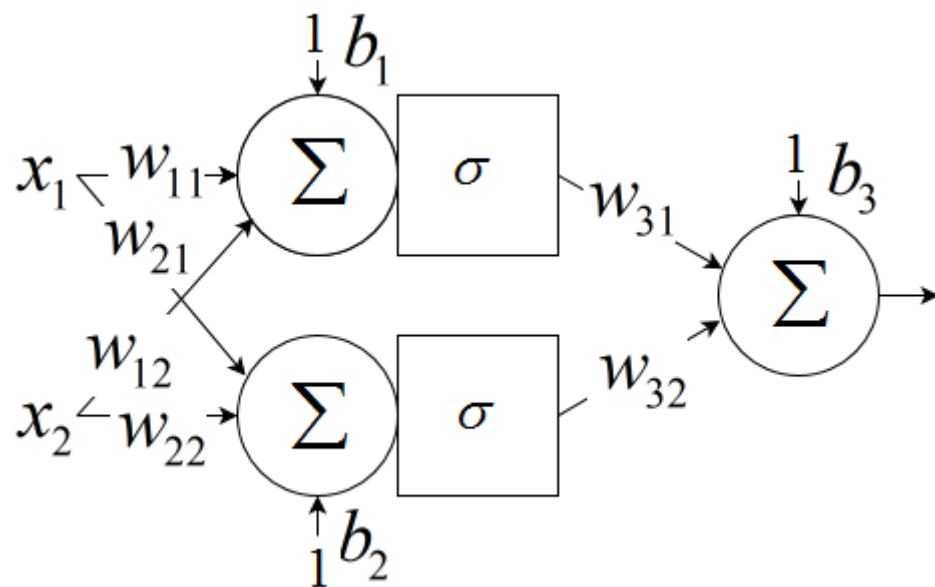
$$\sigma_c(\sigma_c(x_1 + x_2 - 1.5) + \sigma_c(-x_1 - x_2 + 0.5) - 0.5)$$

$$\sigma_c(z) = \frac{1}{1 + e^{-cz}}$$

- Сигмоиду проще обучать – дифференцируемая
 - Есть возможность получать «вероятности»

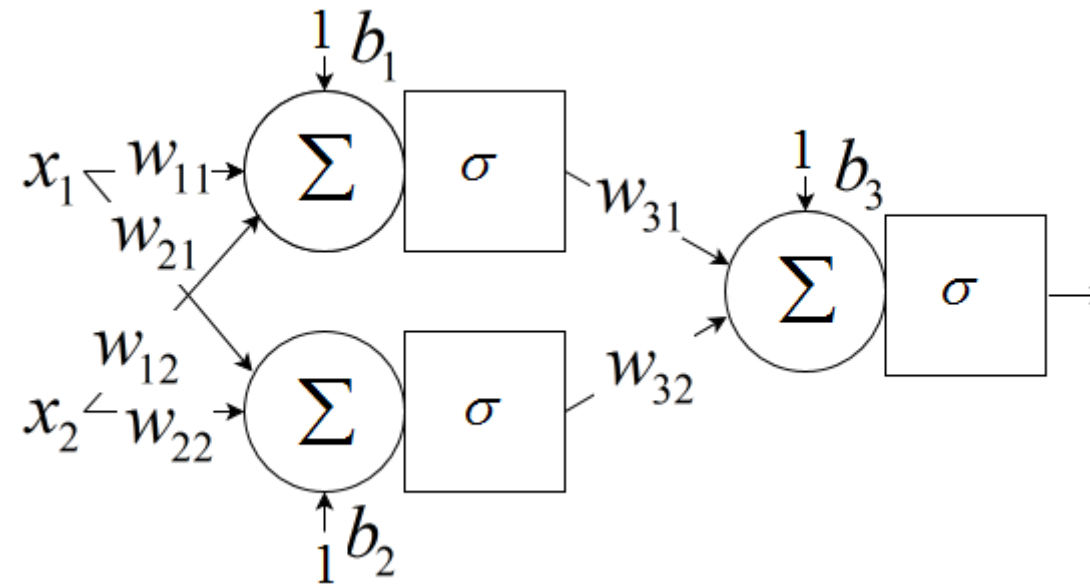
Двуслойная нейронная сеть

Регрессия



$$a = b_3 + w_{31}\sigma(b_1 + w_{11}x_1 + w_{12}x_2) + w_{32}\sigma(b_2 + w_{21}x_1 + w_{22}x_2)$$

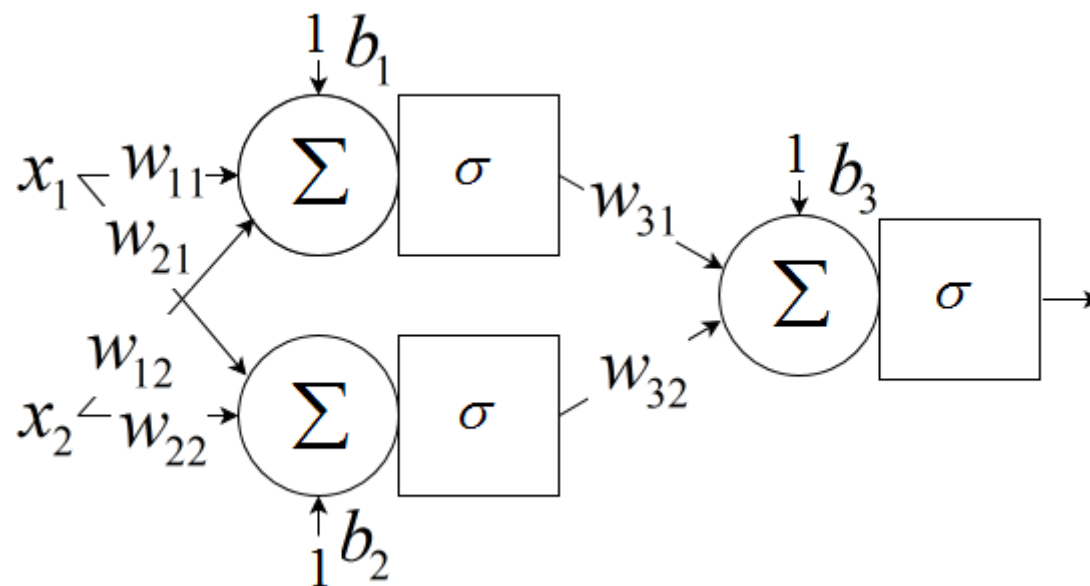
Классификация



$$a = \sigma(b_3 + w_{31}\sigma(b_1 + w_{11}x_1 + w_{12}x_2) + w_{32}\sigma(b_2 + w_{21}x_1 + w_{22}x_2))$$

Такой нейронной сети хватит... (в первом слое м.б. больше нейронов)

Двуслойная нейронная сеть: представление в виде матричных умножений



$$\sigma \left(\begin{bmatrix} w_{31} & w_{32} & b_3 \end{bmatrix} \sigma \left(\begin{bmatrix} w_{11} & w_{12} & b_1 \\ w_{21} & w_{22} & b_2 \\ & & 1 \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \\ 1 \end{pmatrix} \right) \right)$$

$$\sigma \left(\begin{bmatrix} w_{31} & w_{32} \end{bmatrix} \left[\sigma \left(\begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right) \right] + b_3 \right)$$

Двухслойная сеть – изображение

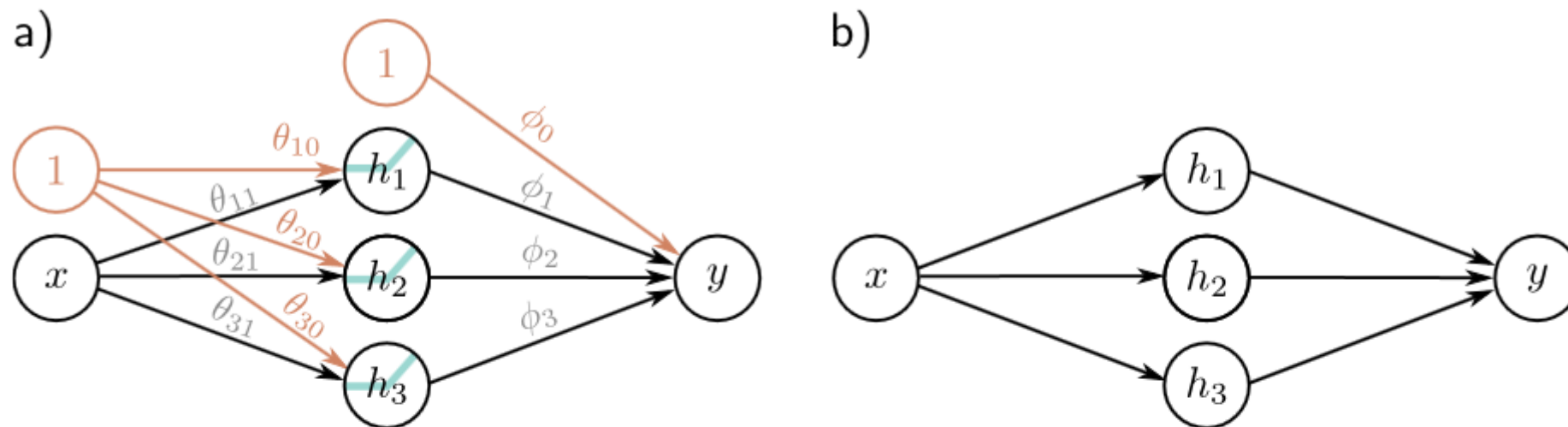


Figure 3.4 Depicting neural networks. a) The input x is on the left, the hidden units h_1, h_2 , and h_3 in the center, and the output y on the right. Computation flows from left to right. The input is used to compute the hidden units, which are combined to create the output. Each of the ten arrows represents a parameter (intercepts in orange and slopes in black). Each parameter multiplies its source and adds the result to its target. For example, we multiply the parameter ϕ_1 by source h_1 and add it to y . We introduce additional nodes containing ones (orange circles) to incorporate the offsets into this scheme, so we multiply ϕ_0 by one (with no effect) and add it to y . ReLU functions are applied at the hidden units. b) More typically, the intercepts, ReLU functions, and parameter names are omitted; this simpler depiction represents the same network.

<https://udlbook.github.io/udlbook/>

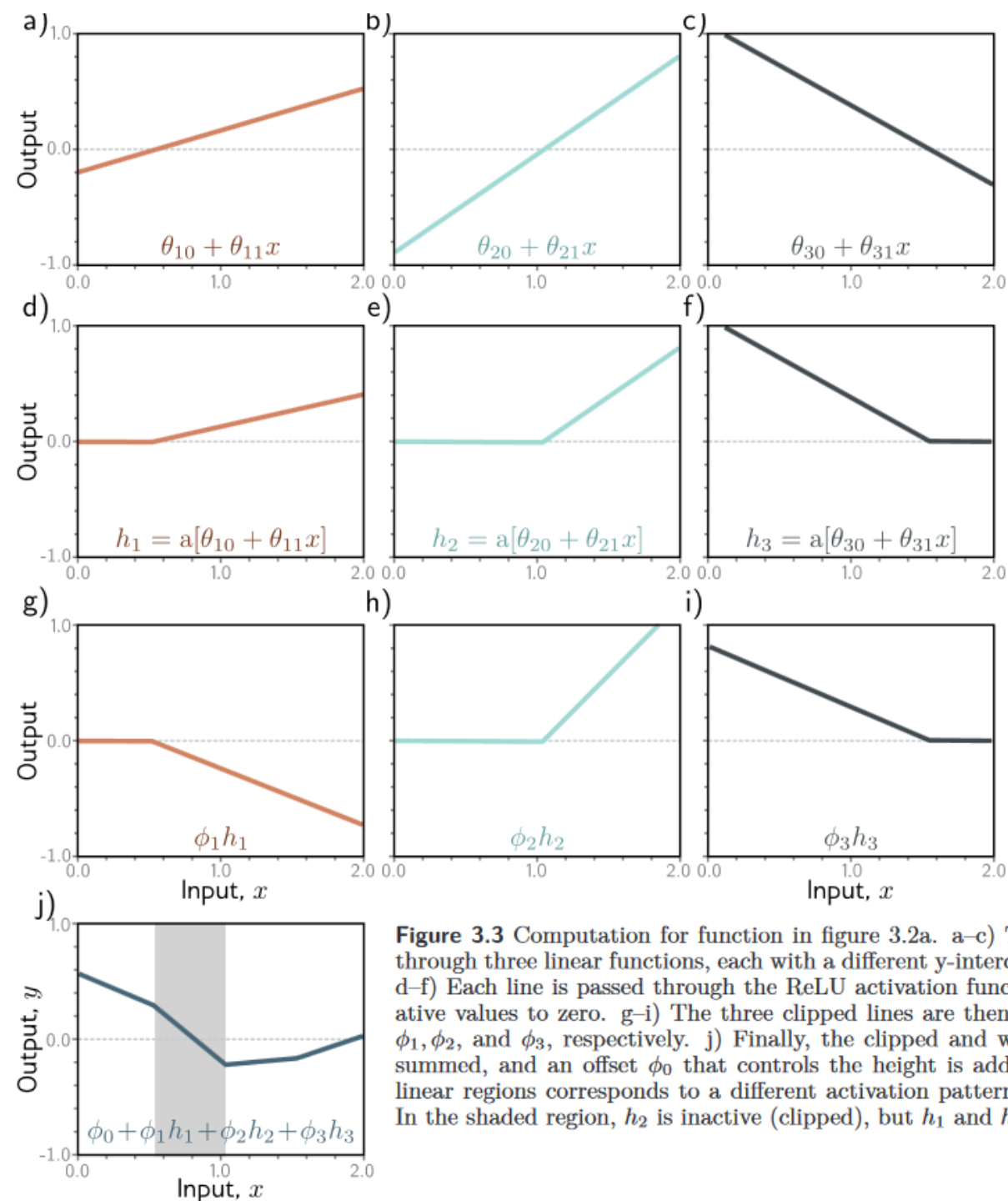


Figure 3.3 Computation for function in figure 3.2a. a–c) The input x is passed through three linear functions, each with a different y-intercept $\theta_{\bullet 0}$ and slope $\theta_{\bullet 1}$. d–f) Each line is passed through the ReLU activation function, which clips negative values to zero. g–i) The three clipped lines are then weighted (scaled) by ϕ_1 , ϕ_2 , and ϕ_3 , respectively. j) Finally, the clipped and weighted functions are summed, and an offset ϕ_0 that controls the height is added. Each of the four linear regions corresponds to a different activation pattern in the hidden units. In the shaded region, h_2 is inactive (clipped), but h_1 and h_3 are both active.

Теорема об универсальной аппроксимации [Hornik, 1991]

Любую непрерывную функцию можно с любой точностью приблизить нейросетью глубины 2 с сигмоидной функцией активации на скрытом слое и линейной функции на выходном слое

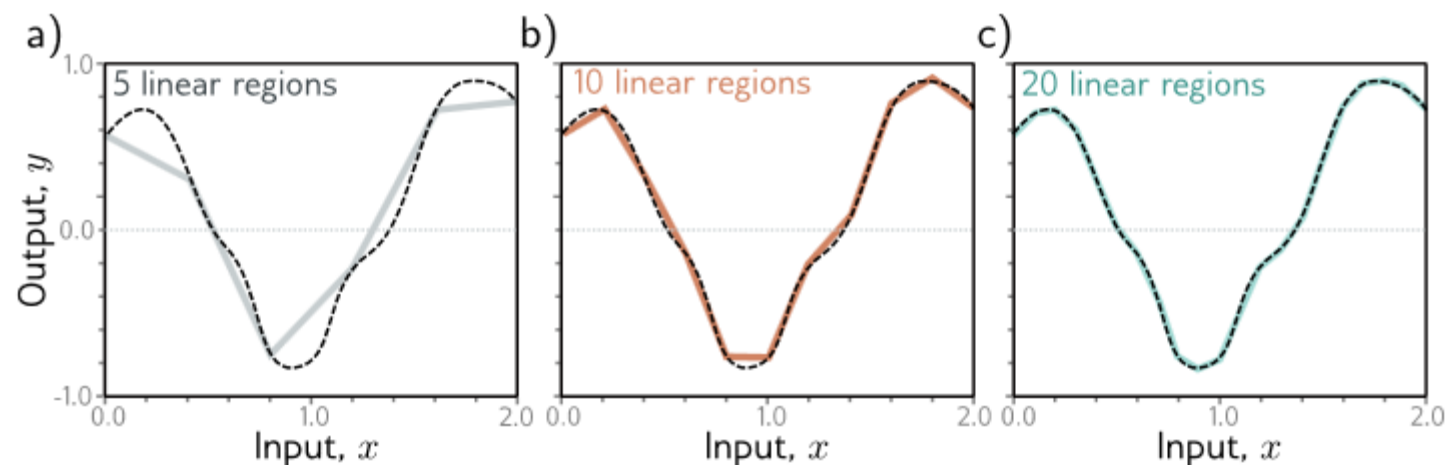


Figure 3.5 Approximation of a 1D function (dashed line) by a piecewise linear model. a–c) As the number of regions increases, the model becomes closer and closer to the continuous function. A neural network with a scalar input creates one extra linear region per hidden unit. The universal approximation theorem proves that, with enough hidden units, there exists a shallow neural network that can describe any given continuous function defined on a compact subset of $\mathbb{R}^{D_i}$ to arbitrary precision.

<https://udlbook.github.io/udlbook/>

Теорема об универсальной аппроксимации Пинкуса

Нейросеть глубины два с фиксированной функцией активации в первом слое и линейной функцией активации во втором может равномерно аппроксимировать (м.б. при увеличении числа нейронов в первом слое) любую непрерывную функцию на компактном множестве тогда и только тогда, когда функция активации неполиномиальная

<http://www2.math.technion.ac.il/~pinkus/papers/neural.pdf>

т.е. функция активации м.б. любая (неполиномиальная), но...

- много нейронов (неизвестно сколько)
- экспоненциальные веса
- сложность обучения

Многослойная нейронная сеть – пример нелинейной модели

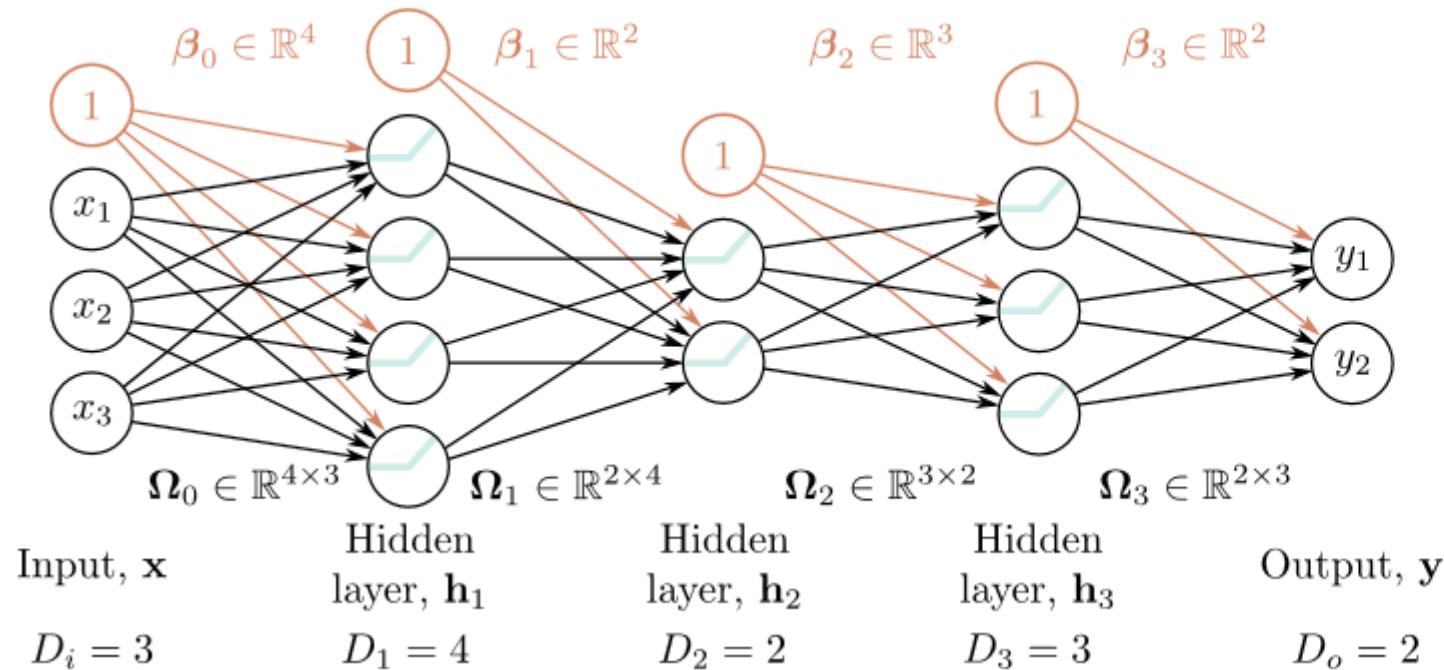


Figure 4.6 Matrix notation for network with $D_i = 3$ -dimensional input $\mathbf{x}$, $D_o = 2$ -dimensional output $\mathbf{y}$, and $K = 3$ hidden layers $\mathbf{h}_1, \mathbf{h}_2$, and $\mathbf{h}_3$ of dimensions $D_1 = 4$, $D_2 = 2$, and $D_3 = 3$ respectively. The weights are stored in matrices Ω_k that multiply the activations from the preceding layer to create the pre-activations at the subsequent layer. For example, the weight matrix Ω_1 that computes the pre-activations at $\mathbf{h}_2$ from the activations at $\mathbf{h}_1$ has dimension 2×4 . It is applied to the four hidden units in layer one and creates the inputs to the two hidden units at layer two. The biases are stored in vectors β_k and have the dimension of the layer into which they feed. For example, the bias vector β_2 is length three because layer $\mathbf{h}_3$ contains three hidden units.

Ориентированный граф вычислений

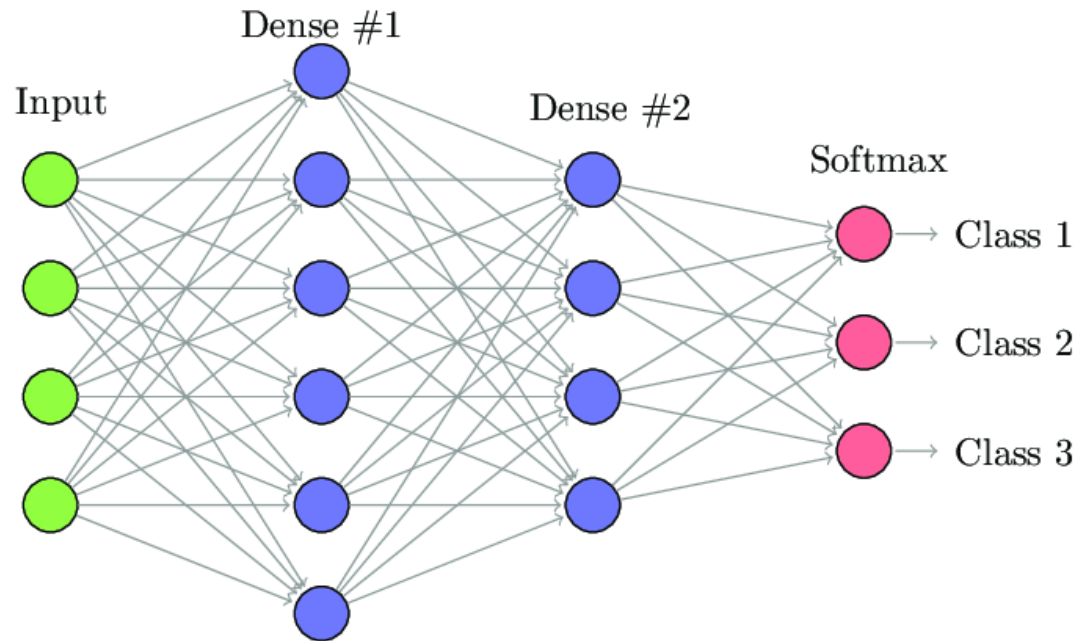
Вершины – переменные или нейроны

Рёбра – зависимости

Сеть прямого распространения – Feedforward Neural Network (FFN) – без циклов
входной слой + один или несколько скрытых слоёв + выходной слой

Важная аналогия

Глубокая НС – последовательное преобразование признакового пространства



$$\varphi_k(W_k \cdot \dots \cdot \varphi_2(W_2 \cdot \varphi_1(W_1 \cdot x + b_1) + b_2) \dots + b_k)$$

иногда пишут по-другому
это корректно?

$$\varphi_k(W_k \cdot \dots \cdot \varphi_2(W_2 \cdot \varphi_1(W_1 \cdot x)) \dots)$$

Сейчас наука DL, в основном, как правильно представлять (преобразовывать) признаковые пространства

увидим потом и в таких моделях, как кодировщик
нельзя просто преобразовывать... надо что-то ещё требовать

Минутка кода

```
class Feedforward(torch.nn.Module):
    def __init__(self, input_size, hidden_size):
        super(Feedforward, self).__init__()
        self.input_size, self.hidden_size = input_size, hidden_size
        self.fc1 = torch.nn.Linear(self.input_size, self.hidden_size, bias=True)
        self.relu = torch.nn.ReLU()
        self.fc2 = torch.nn.Linear(self.hidden_size, 1, bias=False)
        self.sigmoid = torch.nn.Sigmoid()

    def forward(self, x):
        hidden = self.fc1(x)
        relu = self.relu(hidden)
        output = self.fc2(relu)
        output = self.sigmoid(output)
        return output

net = Feedforward(3, 5)
x = torch.tensor([1., 2., 3.])
net(x)

tensor([0.5376], grad_fn=<SigmoidBackward>)
```


Минутка кода

```
net
Feedforward(
  (fc1): Linear(in_features=3, out_features=5, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=5, out_features=1, bias=False)
  (sigmoid): Sigmoid()
)

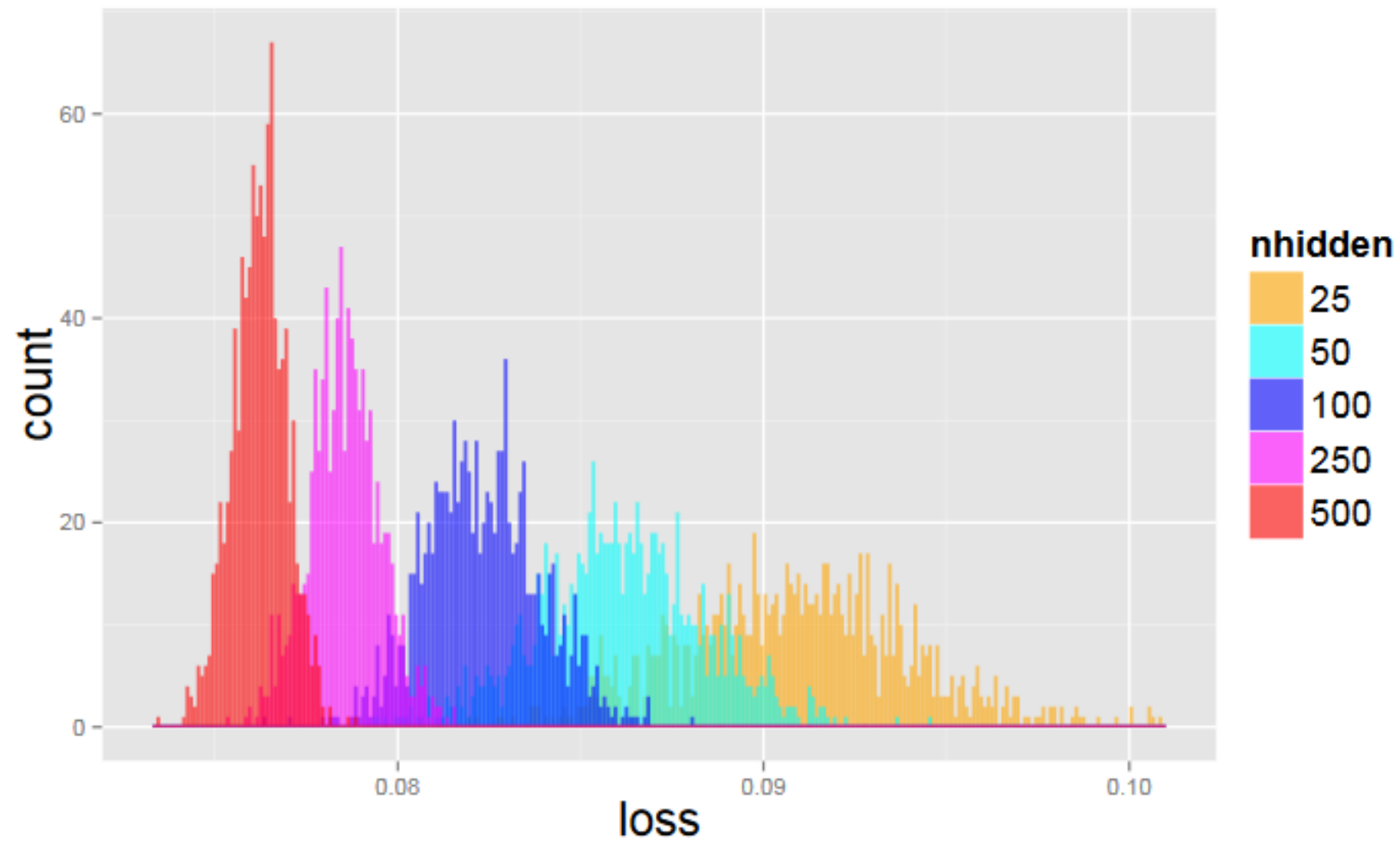
net.fc1.weight.data, net.fc1.bias.data, net.fc2.weight.data # net.fc2.bias.data
(tensor([[ -0.5727,  0.1885, -0.4232],
         [-0.1383, -0.2233, -0.5384],
         [-0.3583,  0.1175, -0.5696],
         [-0.4284,  0.1711,  0.0918],
         [-0.5539, -0.2273,  0.4973]])),
tensor([ 0.5525, -0.4201, -0.5736,  0.0464, -0.0297]),
tensor([[ -0.1064, -0.3029,  0.4057,  0.0953,  0.2823]]))
```

ИЛИ

```
from torch import nn
net = nn.Sequential(nn.Linear(3, 5), nn.ReLU(), nn.Linear(5, 1), nn.Sigmoid())
net(x)
```

как реализовать **Feedforward([3, 4, 5, 3, 1])?**

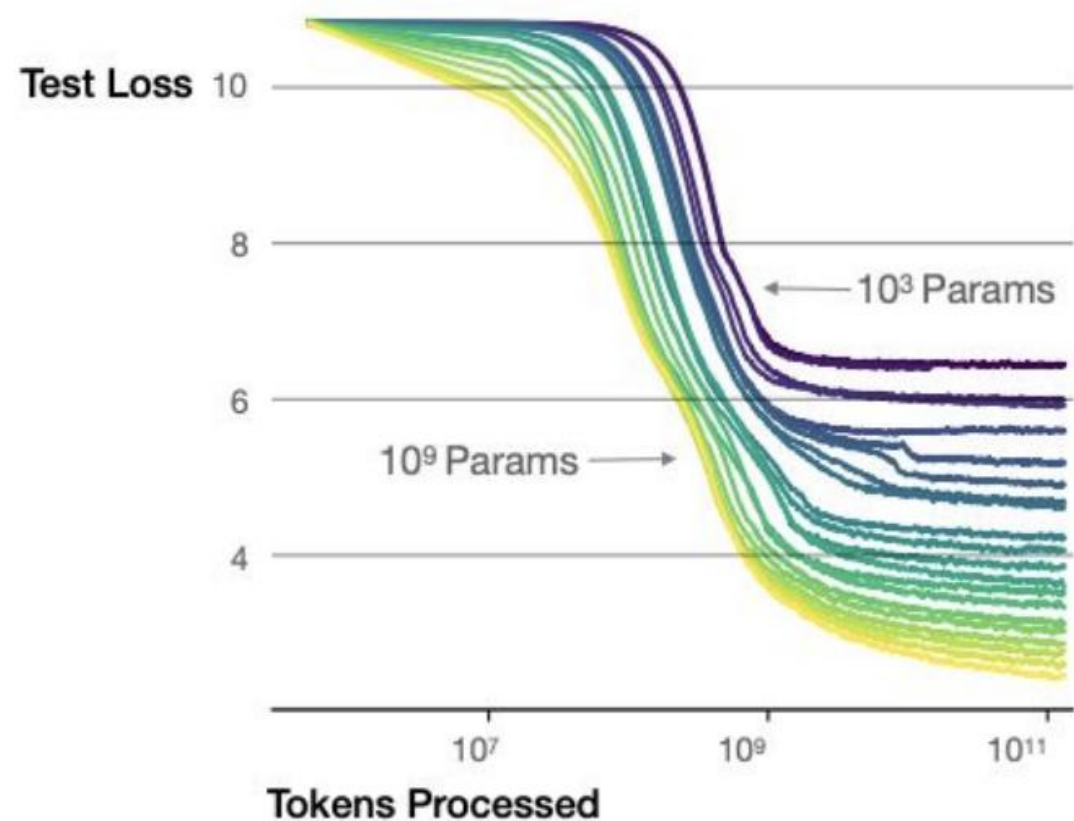
Зачем нужны глубокие нейронные сети



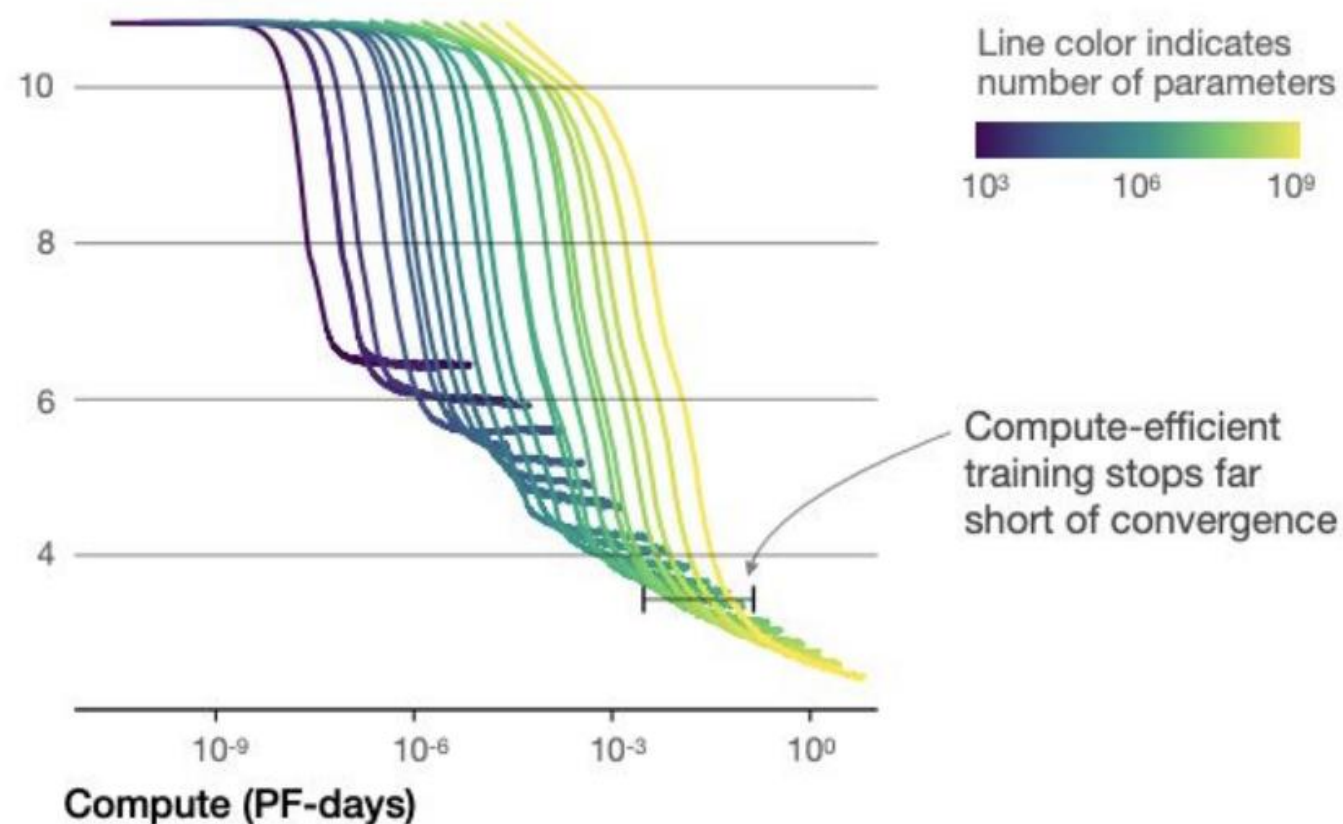
Anna Choromanska, Mikael Henaff, Michael Mathieu, Gérard Ben Arous, Yann LeCun
«The Loss Surfaces of Multilayer Networks» 2015, <https://arxiv.org/abs/1412.0233>

Зачем нужны глубокие нейронные сети

Larger models require **fewer samples** to reach the same performance

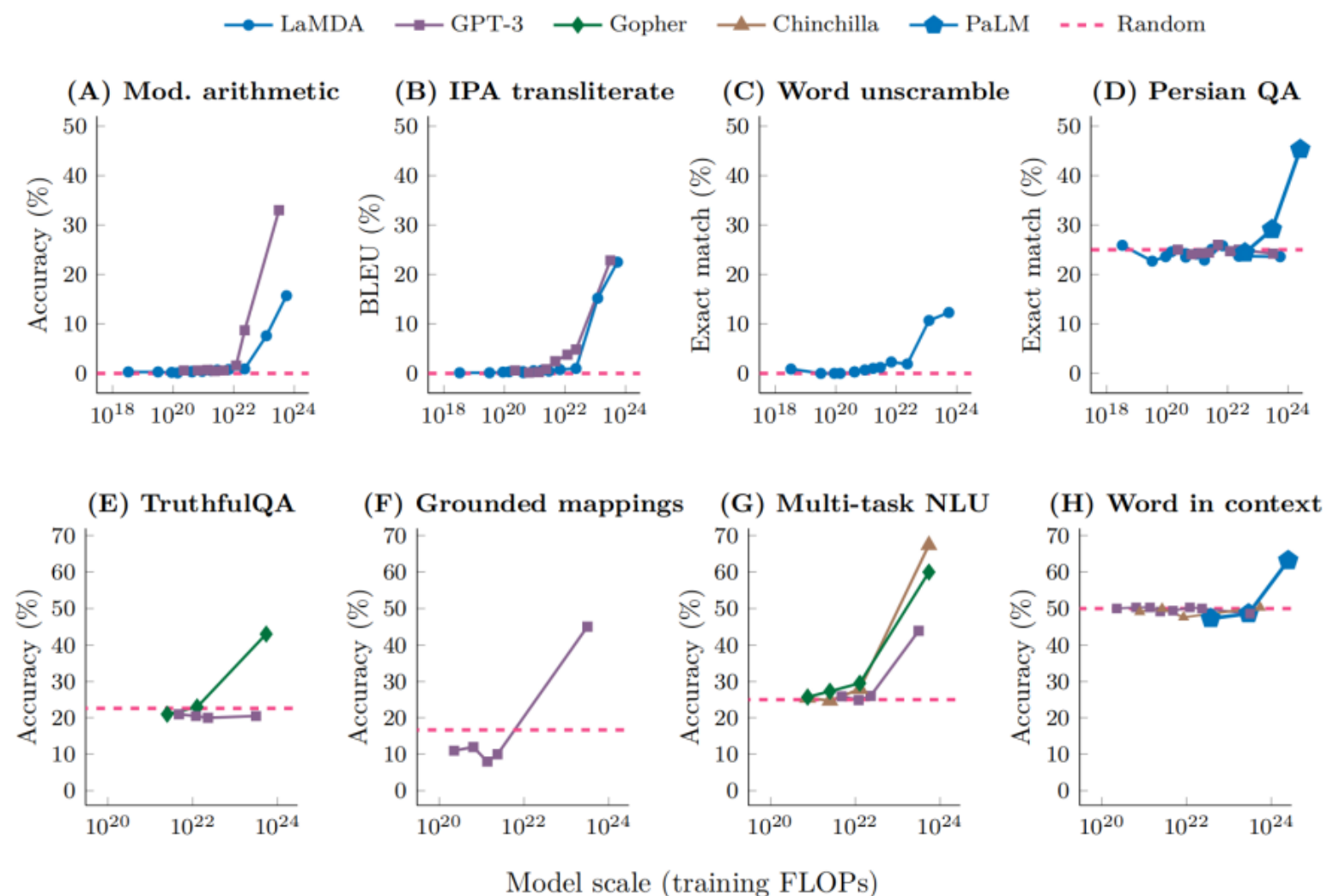


The optimal model size grows smoothly with the loss target and compute budget



Большие модели требуют меньше данных!
stateofai 2020

Зачем нужны глубокие нейронные сети



В больших моделях появляется эмерджентность
(и свойство нельзя предсказать анализом маленьких моделей)

Глубокие нейронные сети

- Много слоёв
- Много данных
- Достаточно вычислительных мощностей

Обучение

Как принято... минимизация регуляризованного эмпирического риска

$$\frac{1}{m} \sum_{i=1}^m [L(a(x_i | w), y_i) + \lambda R(w)] \rightarrow \min_w$$

Задача оптимизации невыпуклая!

«Настройка» нейронной сети – получение весов w

Метод стохастического градиента (SGD)

$$w^{(t+1)} = w^{(t)} - \eta \nabla [L(a(x_i | w^{(t)}), y_i) + \lambda R(w^{(t)})]$$

т.к. очень много слагаемых... и так быстрее;)

Метод стохастического градиента

1. Случайная инициализация весов $w^{(0)} \sim \text{norm}(0, \sigma^2)$

2. Цикл по t до сходимости

2.1. Выбираем случайный объект x_i

2.2. Вычисляем градиент

$$\nabla[L(a(x_i | w^{(t)}), y_i) + \lambda R(w^{(t)})]$$

2.3. Адаптация весов

$$w^{(t+1)} = w^{(t)} - \eta \nabla[L(a(x_i | w^{(t)}), y_i) + \lambda R(w^{(t)})]$$

почему такая инициализация?

Функции ошибки

Классификация – logloss (CrossEntropyLoss)

$$L((a_1, \dots, a_l), y) = -\log \frac{\exp(a_y)}{\sum_{j=1}^l \exp(a_j)} = -a_y + \log \sum_{j=1}^l \exp(a_j)$$

такая реализация не очень «устойчива»
(сумма больших экспонент, разница близких чисел)

Часто при реализации делают так:

$$-a_y + \max \{a_j\} + \log \left(\sum_{j=1}^l \exp(a_j - \max \{a_j\}) \right)$$

Минутка кода

```
input = torch.randn(3, 5, requires_grad=True) # m x l
target = torch.randint(5, (3,), dtype=torch.int64)
loss = torch.nn.functional.cross_entropy(input, target)
loss.backward()
```

`input, target, loss`

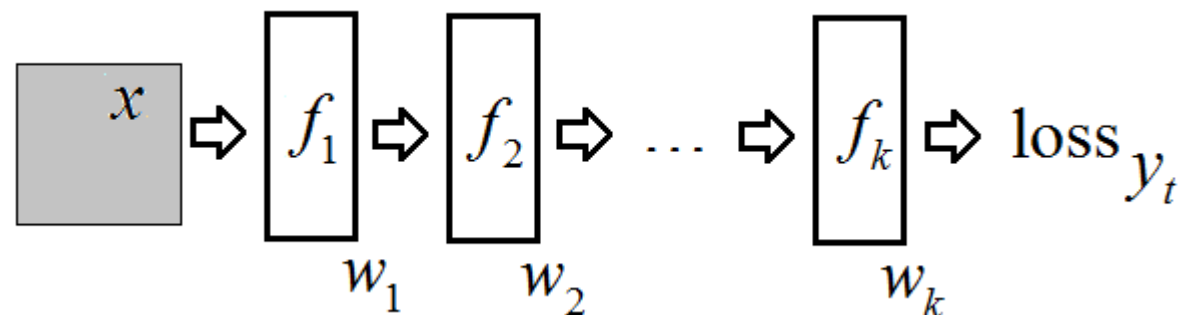
```
(tensor([[-0.0691, -1.5644, -0.3935,  0.2211,  1.3458],
         [-2.3893,  1.2955,  0.0244, -2.0838,  1.0790],
         [-0.6185, -0.0333,  0.2944, -0.6185,  0.5702]]), requires_grad=True),
tensor([4, 3, 1]),
tensor(2.1341, grad_fn=<NllLossBackward>))
```

перечисляются номера классов,
но можно и матрицу с вероятностями для soft-labels

Обратное распространение градиента (Backpropagation)

Идея: вычисление производной сложной функции

$$L = \sum_t \text{loss}(y_t, f(x_t, w))$$



$$f(x, w) = f_k(\dots f_2(f_1(x, w_1), w_2) \dots)$$

у нас суперпозиция

чуть пока упростим

$$L \sim L(f_k(\dots f_2(f_1(w)) \dots))$$

$$\nabla L = \frac{\partial f_1}{\partial w}(\mathbf{w}) \cdot \frac{\partial f_2}{\partial f_1}(f_1(\mathbf{w})) \cdot \dots \cdot \frac{\partial L}{\partial f_k}(f_k(\dots f_2(f_1(w)) \dots))$$

(синим – в каких точках производные / градиенты / матрицы Якоби)
надо знать значения «внутри суперпозиции»

Автоматическое дифференцирование

Прямое распространение в $L \sim L(f_k(\dots f_2(f_1(w))\dots))$

$$f_1(w) \rightarrow f_2(f_1(w)) \rightarrow f_3(f_2(f_1(w))) \rightarrow \dots \rightarrow f_k(\dots f_2(f_1(w))\dots)$$

Обратное распространение в $L \sim L(f_k(\dots f_2(f_1(w))\dots))$

$$\begin{aligned} \frac{\partial L}{\partial f_k}(f_k(\dots f_2(f_1(w))\dots)) &\rightarrow \frac{\partial L}{\partial f_{k-1}} = \frac{\partial L}{\partial f_k} \frac{\partial f_k}{\partial f_{k-1}}(f_{k-1}(\dots f_2(f_1(w))\dots)) \rightarrow \\ &\dots \\ \rightarrow \frac{\partial L}{\partial f_1} &= \frac{\partial L}{\partial f_k} \frac{\partial f_k}{\partial f_{k-1}} \dots \frac{\partial f_3}{\partial f_2} \frac{\partial f_2}{\partial f_1}(f_1(w)) \rightarrow \frac{\partial L}{\partial w} = \frac{\partial L}{\partial f_k} \dots \frac{\partial f_2}{\partial f_1} \frac{\partial f_1}{\partial w}(w) \end{aligned}$$

синее – вычисленное на предыдущем шаге

красное – вычисленное на прямом проходе (не всегда указано)

но всё происходит послойно!

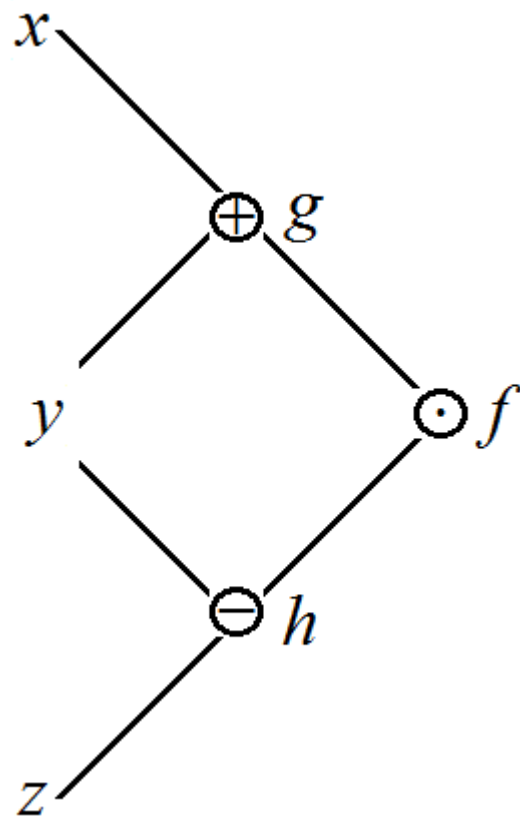
Нейросеть – вычислительный граф

Другой взгляд на НС
вершины – переменные (входные, внутренние, выходные)
рёбра – зависимости (+ веса)
слой – операция

тут м.б. более широкое понятие слоя:
линейная комбинация,
нелинейность,
...

Вычисление градиента на графе

$$f = (x + y) \cdot (y - z)$$



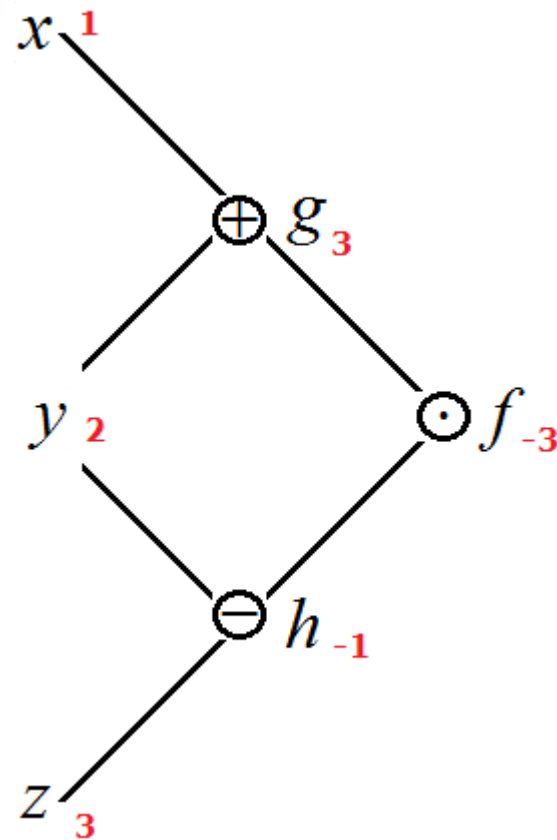
$$f(x, y, z) = \underbrace{(x + y)}_{g(x, y)} \cdot \underbrace{(y - z)}_{h(y, z)}$$

Как проводится вычисление функции?

$$x, y, z = 1, 2, 3$$

Вычисление градиента на графе

$$f = (x + y) \cdot (y - z)$$



$$f(x, y, z) = \underbrace{(x + y)}_{g(x, y)} \cdot \underbrace{(y - z)}_{h(y, z)}$$

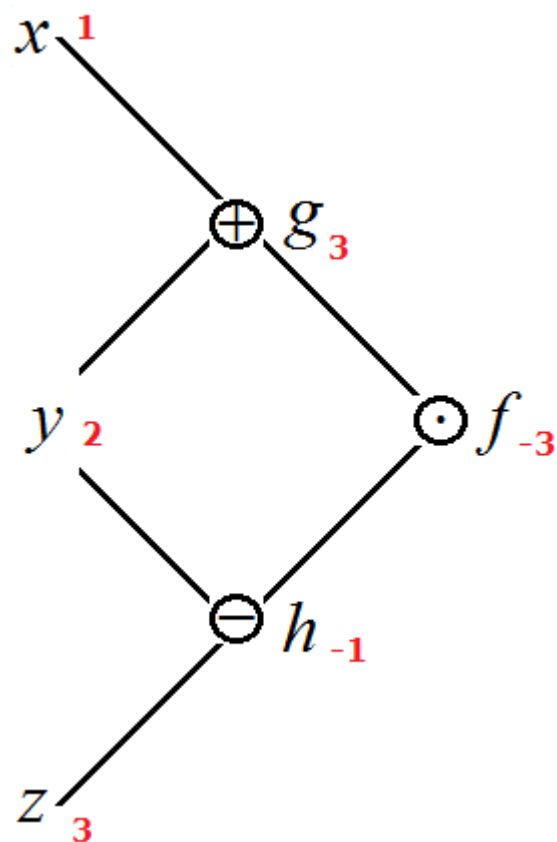
Как проводится вычисление функции?

$$x, y, z = 1, 2, 3$$

«Прямой ход»

Вычисление градиента на графе

$$f = (x + y) \cdot (y - z)$$



$$f(x, y, z) = \underbrace{(x + y)}_{g(x, y)} \cdot \underbrace{(y - z)}_{h(y, z)}$$

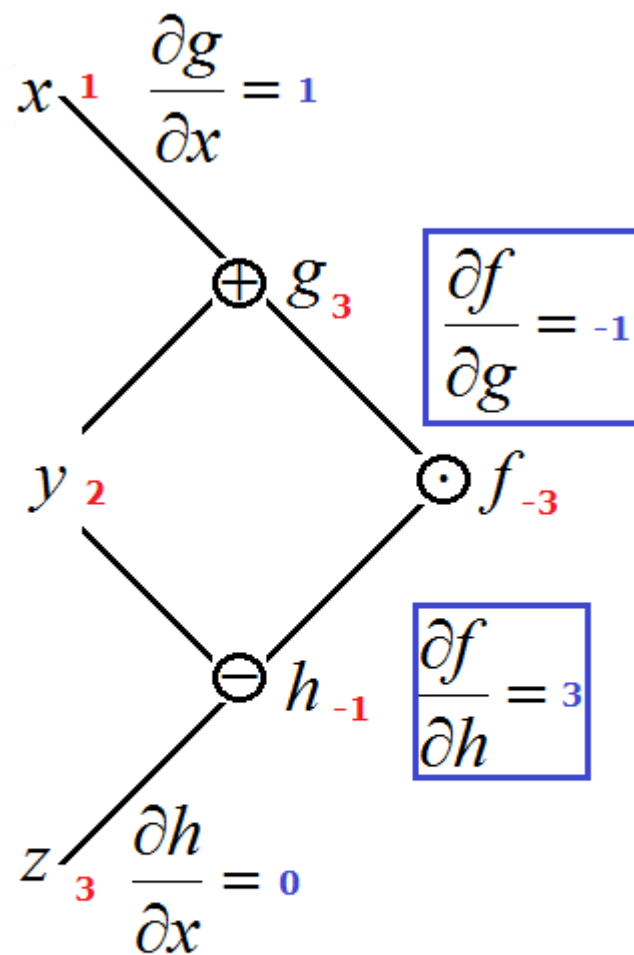
Как проводится вычисление производных?

$$\frac{\partial f}{\partial g} = h, \quad \frac{\partial f}{\partial h} = g$$

$$\frac{\partial g}{\partial x} = 1, \quad \frac{\partial h}{\partial x} = 0$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial x} + \frac{\partial f}{\partial h} \frac{\partial h}{\partial x}$$

Вычисление градиента на графе



$$\frac{\partial f}{\partial x} = \boxed{\frac{\partial f}{\partial g}} \frac{\partial g}{\partial x} + \boxed{\frac{\partial f}{\partial h}} \frac{\partial h}{\partial x}$$

$$f(x, y, z) = \underbrace{(x + y)}_{g(x, y)} \cdot \underbrace{(y - z)}_{h(y, z)}$$

Как проводится вычисление производных?

«Обратный ход»

Минутка кода: PyTorch

```

import torch
from torch.autograd import Variable

# переменные
x = Variable(torch.Tensor([1]), requires_grad=True)
y = Variable(torch.Tensor([2]), requires_grad=True)
z = Variable(torch.Tensor([3]), requires_grad=True)
# теперь так
x = torch.tensor([1.], requires_grad=True)
y = torch.tensor([2.], requires_grad=True)
z = torch.tensor([3.], requires_grad=True)
f = (x + y) * (y - z) # прямой проход - вычисление

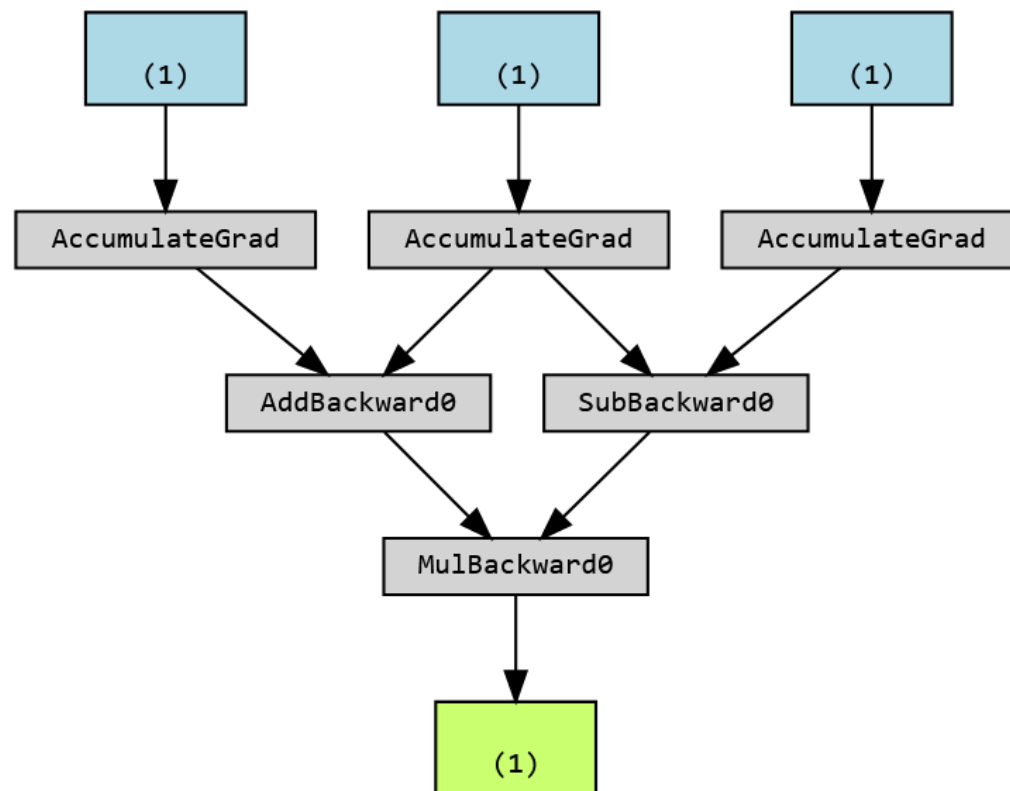
f.backward() # обратный проход - выч. производных
x.grad, y.grad, z.grad # производные

(tensor([-1.]), tensor([2.]), tensor([-3.]))

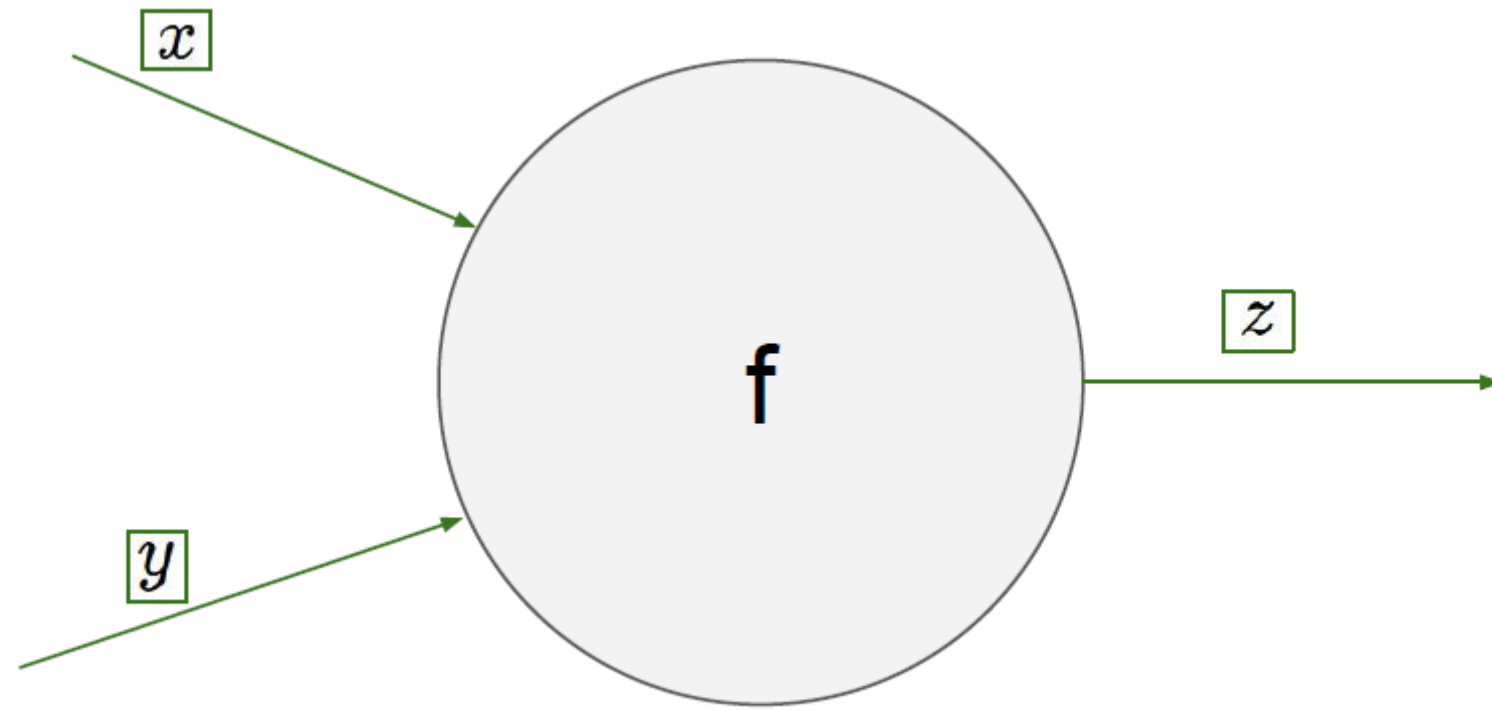
from torchviz import make_dot

make_dot(f)

```



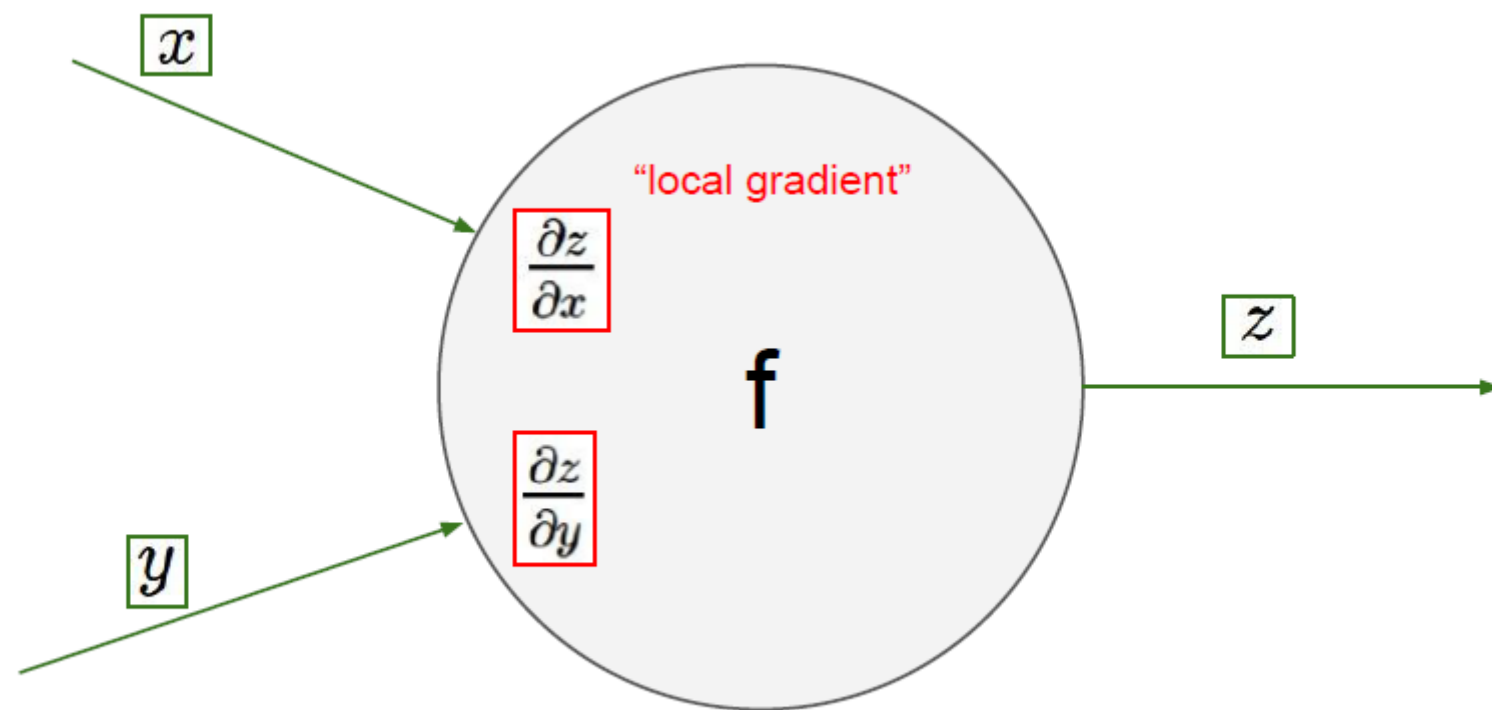
Обратное распространение градиента (Backpropagation)



<http://cs231n.stanford.edu/2017/index.html>

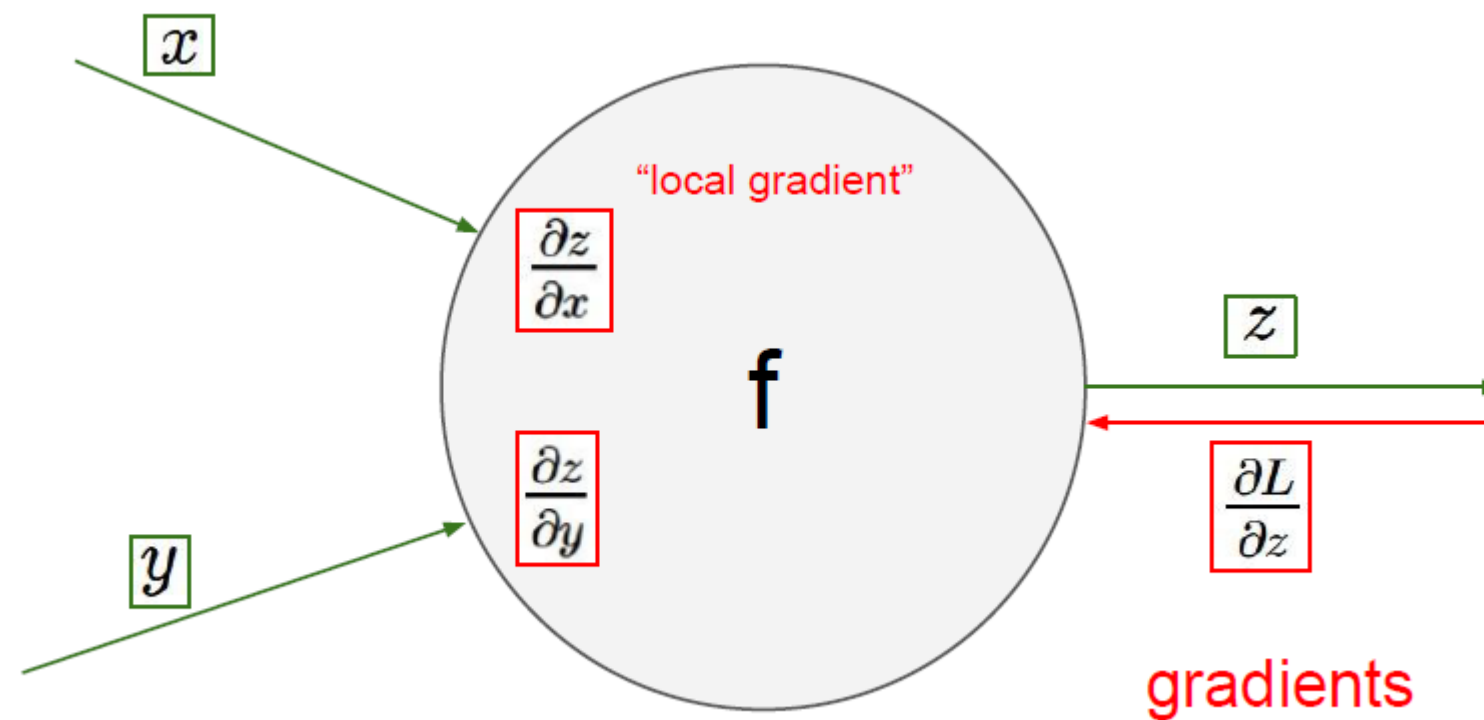
Обратное распространение = SGD + дифференцирование сложных функций

Обратное распространение градиента (Backpropagation)



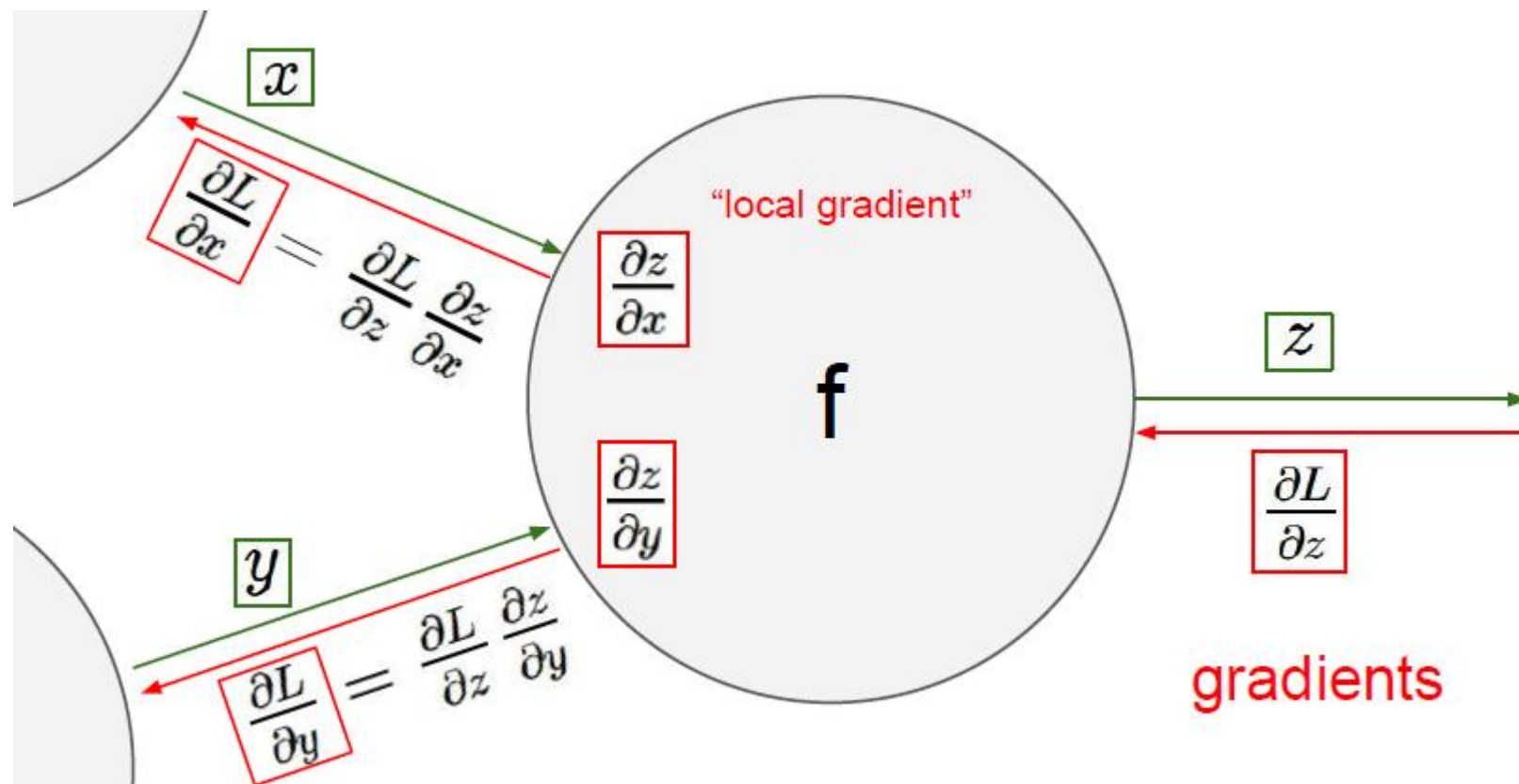
<http://cs231n.stanford.edu/2017/index.html>

Обратное распространение градиента (Backpropagation)



<http://cs231n.stanford.edu/2017/index.html>

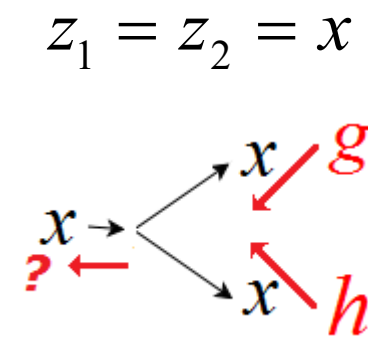
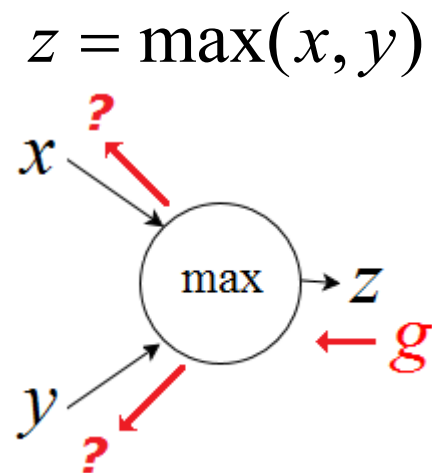
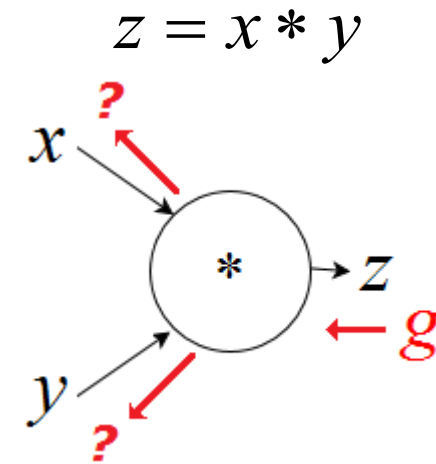
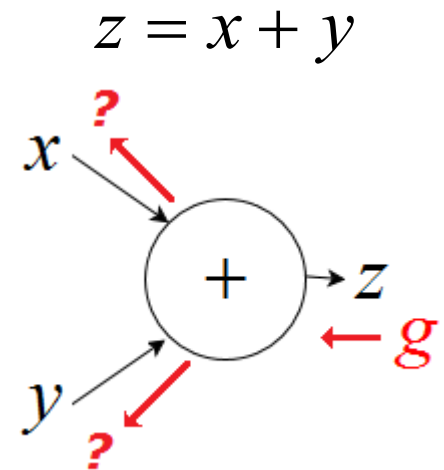
Обратное распространение градиента (Backpropagation)



<http://cs231n.stanford.edu/2017/index.html>

Прохождение градиента через гейты

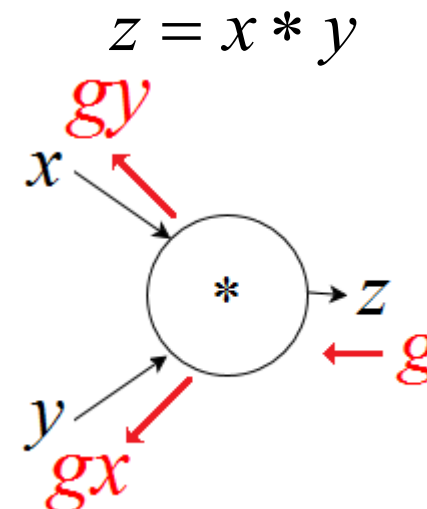
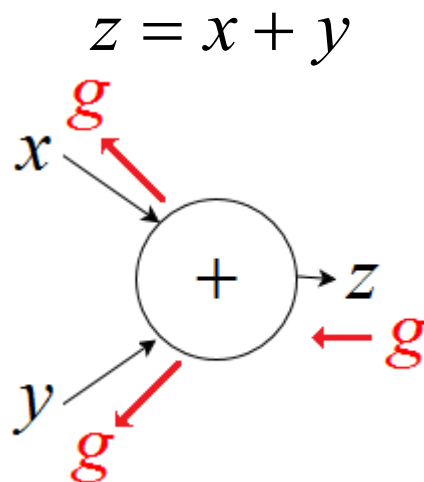
Как здесь проходит градиент?



Прохождение градиента через гейты

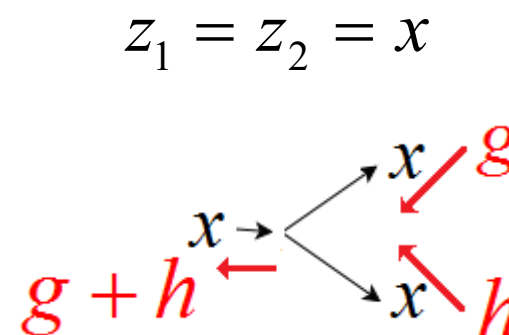
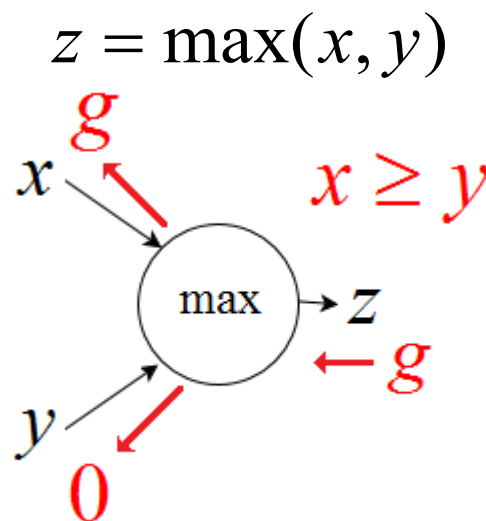
Как здесь проходит градиент?

$$\begin{aligned}\frac{\partial L}{\partial x} &= \frac{\partial L}{\partial z} \frac{\partial z}{\partial x} = \\ &= \textcolor{red}{g} \frac{\partial(x+y)}{\partial x} = \textcolor{red}{g}\end{aligned}$$



$$\begin{aligned}\frac{\partial L}{\partial x} &= \frac{\partial L}{\partial z} \frac{\partial z}{\partial x} = \\ &= \textcolor{red}{g} \frac{\partial(xy)}{\partial x} = \textcolor{red}{gy}\end{aligned}$$

$$\begin{aligned}\frac{\partial L}{\partial x} &= \frac{\partial L}{\partial z} \frac{\partial z}{\partial x} = \\ &= \textcolor{red}{g} \frac{\partial \max(x, y)}{\partial x} = \begin{cases} \textcolor{red}{g}, & x > y \\ 0, & y \geq x \end{cases}\end{aligned}$$



Подумать, почему так? Потом станет яснее...

А что при $x=y$?!

Напоминание для векторного дифференцирования

производная

$$x \in \mathbb{R}, z \in \mathbb{R} \Rightarrow$$

$$\frac{\partial z}{\partial x} \in \mathbb{R}$$

градиент

$$x \in \mathbb{R}^n, z \in \mathbb{R} \Rightarrow$$

$$\frac{\partial z}{\partial x} = \left(\frac{\partial z}{\partial x_1}, \dots, \frac{\partial z}{\partial x_n} \right)^T \in \mathbb{R}^n$$

матрица Якоби

$$x \in \mathbb{R}^n, z \in \mathbb{R}^m \Rightarrow$$

$$\frac{\partial z}{\partial x} = \left\| \frac{\partial z_j}{\partial x_i} \right\|_{n \times m} \in \mathbb{R}^{n \times m}$$

в вики матрица Якоби по-другому (транспонирована)

$$\frac{\partial z}{\partial x} = \left\| \begin{array}{ccc} \frac{\partial z_1}{\partial x_1} & \dots & \frac{\partial z_m}{\partial x_1} \\ \dots & \dots & \dots \\ \frac{\partial z_1}{\partial x_n} & \dots & \frac{\partial z_m}{\partial x_n} \end{array} \right\|_{n \times m} \in \mathbb{R}^{n \times m}$$

Прохождение градиента через гейты – пример

меняем порядок множителей, чтобы произведения совпадали по размерам

«ReLU-слой»: $z = \max(x, 0)$, $x \in \mathbb{R}^n$

$$\underbrace{\frac{\partial z}{\partial x}}_{n \times n} \underbrace{\frac{\partial L}{\partial z}}_{n \times 1} = \frac{\partial z}{\partial x} \frac{\partial L}{\partial z} = \begin{bmatrix} I[x_1 > 0] & 0 & \dots & 0 \\ 0 & I[x_2 > 0] & \ddots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & I[x_n > 0] \end{bmatrix} \frac{\partial L}{\partial z} = \text{diag}(I[x > 0]) \frac{\partial L}{\partial z}$$

Линейный слой: $z = Wx$, $z \in \mathbb{R}^m, x \in \mathbb{R}^n, W \in \mathbb{R}^{m \times n}$

$$\frac{\partial z}{\partial x} = \frac{\partial Wx}{\partial x} = \frac{\partial \begin{bmatrix} w_{11}x_1 + \dots + w_{1n}x_n \\ \dots \\ w_{m1}x_1 + \dots + w_{mn}x_n \end{bmatrix}}{\partial x} = \begin{bmatrix} w_{11} & \dots & w_{1n} \\ \dots & \dots & \dots \\ w_{m1} & \dots & w_{mn} \end{bmatrix} = W^T$$

Прохождение градиента через гейты

Линейный слой + активация

$$z = \text{ReLU}(Wx), \quad z \in \mathbb{R}^m, x \in \mathbb{R}^n, W \in \mathbb{R}^{m \times n}$$

$$\frac{\partial z}{\partial x} = \frac{\partial h}{\partial x} \frac{\partial z}{\partial h} \bigg|_{h=Wx} = W_{n \times m}^T \underbrace{\text{diag}(\nabla \text{ReLU}(Wx))}_{I[Wx > 0]}_{m \times m}$$

Несколько слоёв...

$$z = \text{ReLU}(W_k (\dots W_2 \text{ReLU}(W_1 x)))$$

$$\frac{\partial z}{\partial x} = W_1^T \text{diag}(\nabla \text{ReLU}(Wx)) \cdot W_2^T \text{diag}(\nabla \text{ReLU}(\cdot)) \cdot \dots \cdot W_k^T \text{diag}(\nabla \text{ReLU}(\cdot))$$

Обратное распространение – перемножение матриц Якоби

При некоторых условиях (**каких?**) $\frac{\partial z}{\partial x} = (W^T)^k$

Прохождение градиента через гейты

$$L = L(z) \in \mathbb{R}, z = Wx, \quad z \in \mathbb{R}^m, x \in \mathbb{R}^n, W \in \mathbb{R}^{m \times n}$$

Берём производные по матрице весов

$$\frac{\partial L}{\partial W} = \frac{\frac{\partial L}{\partial z}}{\frac{\partial z}{\partial W}} = \frac{\frac{\partial L}{\partial z}}{\frac{\partial z}{\partial W}} x^T = \nabla_z L \cdot x^T$$

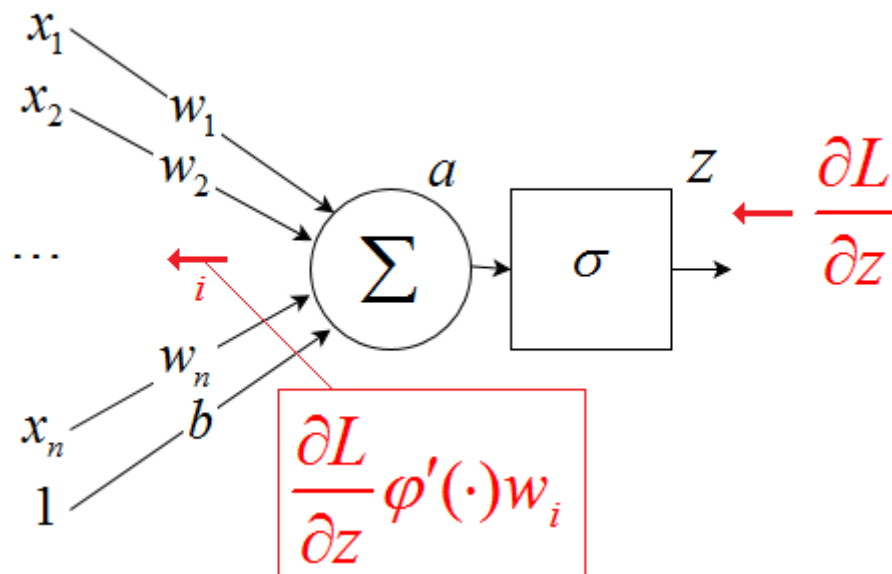
$m \times 1$ $1 \times n$

Можно убедиться, что это так...

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial z}{\partial w_{ij}} \frac{\partial L}{\partial z} = \underbrace{(0, \dots, 0, x_j, 0, \dots, 0)}_i \cdot \nabla_z L = (\nabla_z L)_{[i]} \cdot x_j$$

Здесь немного шульмуем... производная по $m \times n$ -матрице – $m \times n$ -матрица

Прохождение градиента через нейрон



$$z = \varphi(w_0 + w_1x_1 + \dots + w_nx_n)$$

для распространения градиента

$$\frac{\partial L}{\partial x_i} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial x_i} = \frac{\partial L}{\partial z} \varphi'(\cdot) w_i$$

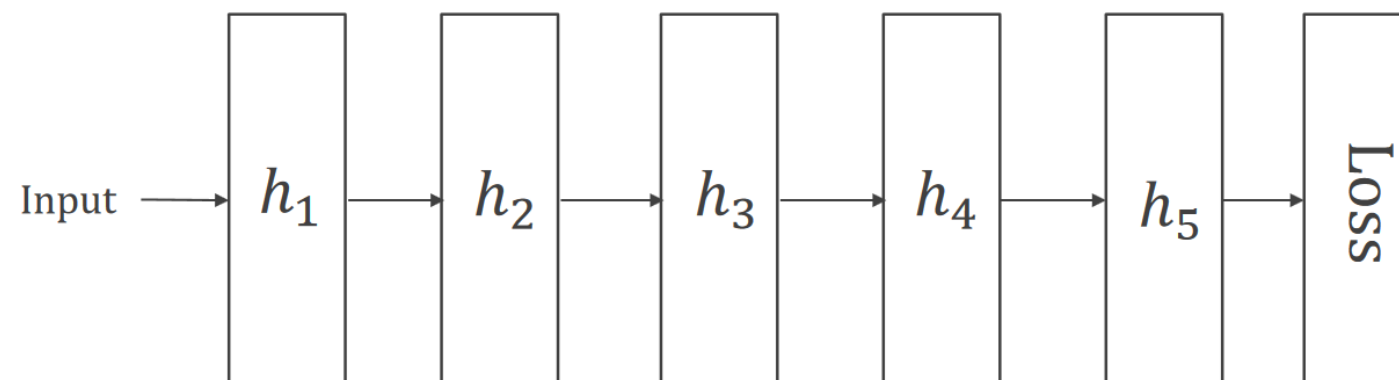
для корректировки весов

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial w_i} = \frac{\partial L}{\partial z} \varphi'(\cdot) x_i$$

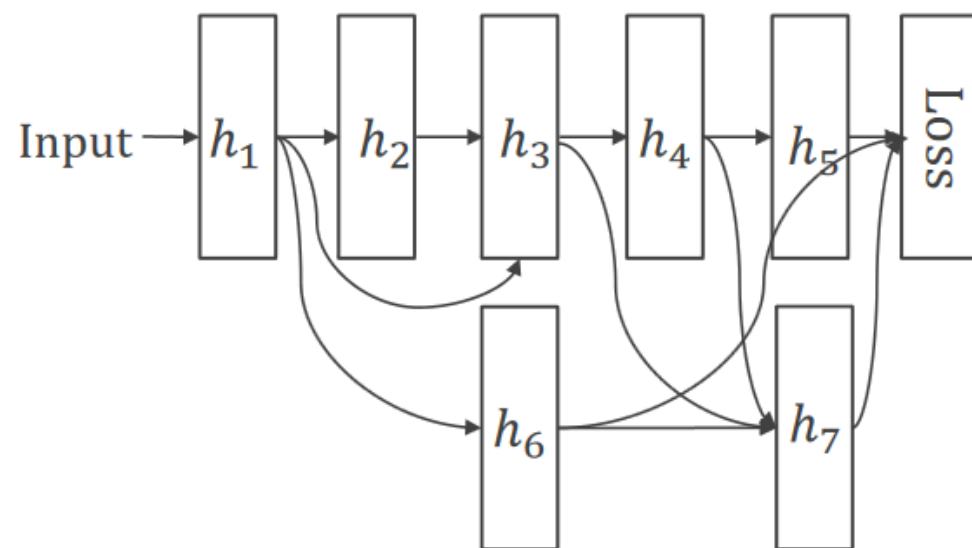
при прохождении градиента через нейрон
происходит умножение на производную функции активации

Графы вычислений могут быть разные

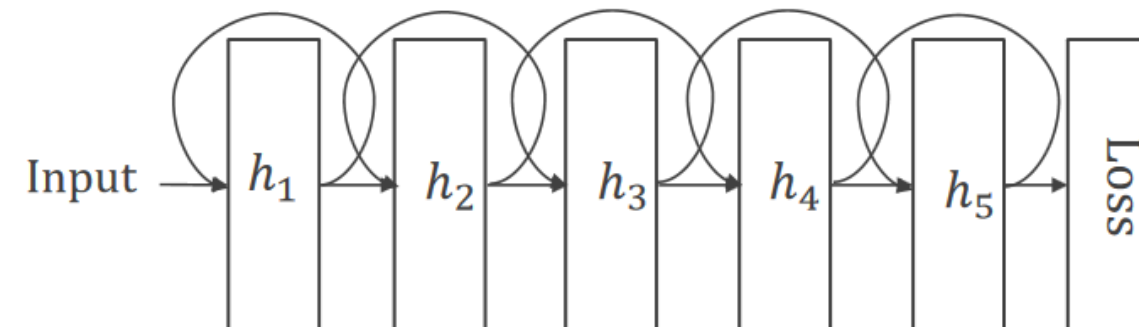
Forward connections (Feedforward architecture)



Interweaved & skip connections



Loopy connections (must be unfolded)



Проблема затухания градиента

Производная сигмоиды при «насыщенном сигнале» близка к нулю

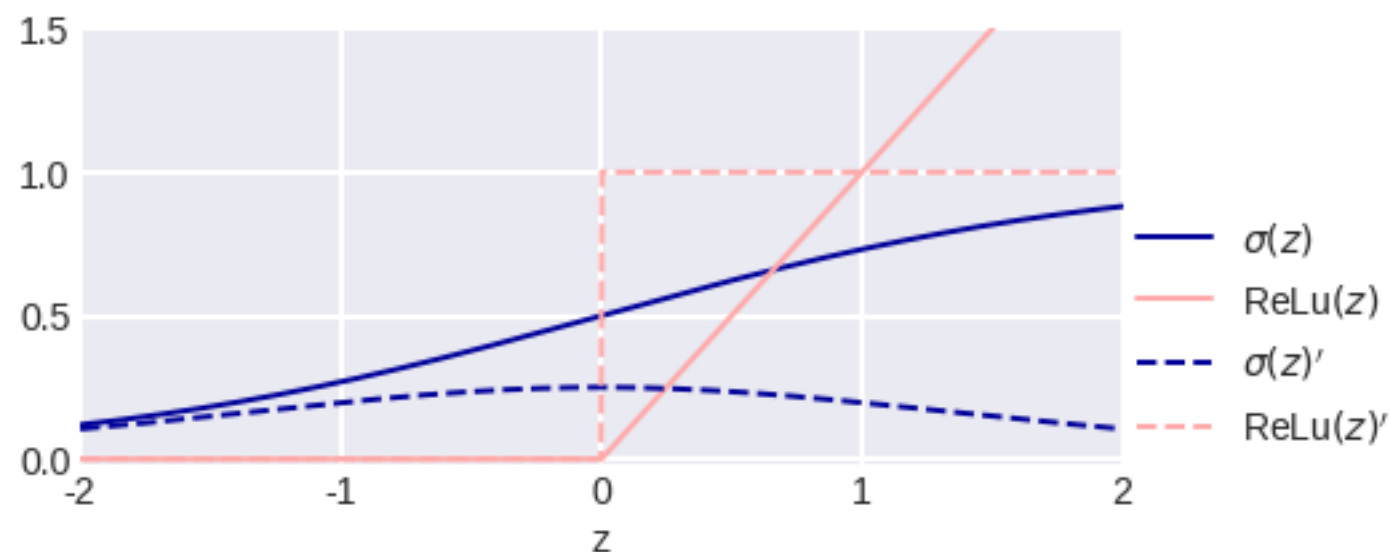
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{\partial \sigma(z)}{\partial z} = \sigma(z)(1 - \sigma(z))$$

ReLU = Rectified Linear Unit

$$\text{ReLU}(z) = \max(0, z)$$

$$\frac{\partial \text{ReLU}(z)}{\partial z} = I[z > 0]$$



Что плохого в сигмоиде

- «убивает» градиенты
- выходы не отцентрированы (легко устранить $\rightarrow \tanh$)
- вычисление экспоненты всё-таки дорого...

Что хорошего в ReLU

- быстро вычисляется
- есть теоретические / биологические обоснования
- зоны константного градиента и обнуления (разреженное решение)

«Dead Neurons»

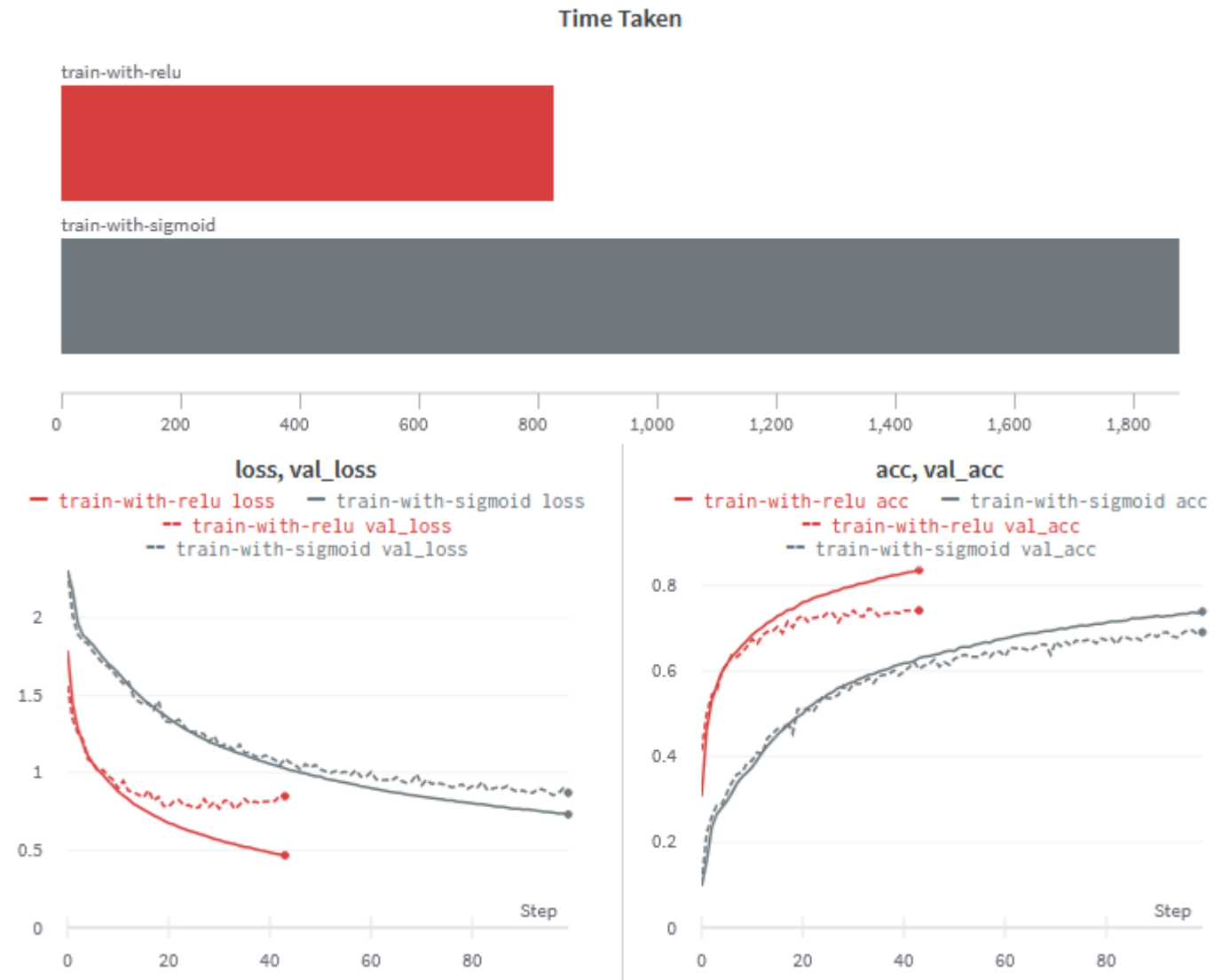
Мёртвые нейроны – выдают ноль на любом объекте выборки

Их можно найти

В ReLU есть опасность их появления

- Но вероятность мала!
- При обучении их число уменьшается
- В глубоких сетях с ReLU они, действительно, могут стать проблемой

Sigmoid vs ReLU



<https://wandb.ai/ayush-thakur/dl-question-bank/reports/ReLU-vs-Sigmoid-Function-in-Deep-Neural-Networks-Why-ReLU-is-so-Prevalent--VmlldzoyMDk0MzI>

Ещё функции активации

LeakyReLU

$$\text{LeakyReLU}(z) = \max(0.1z, z)$$

Softplus

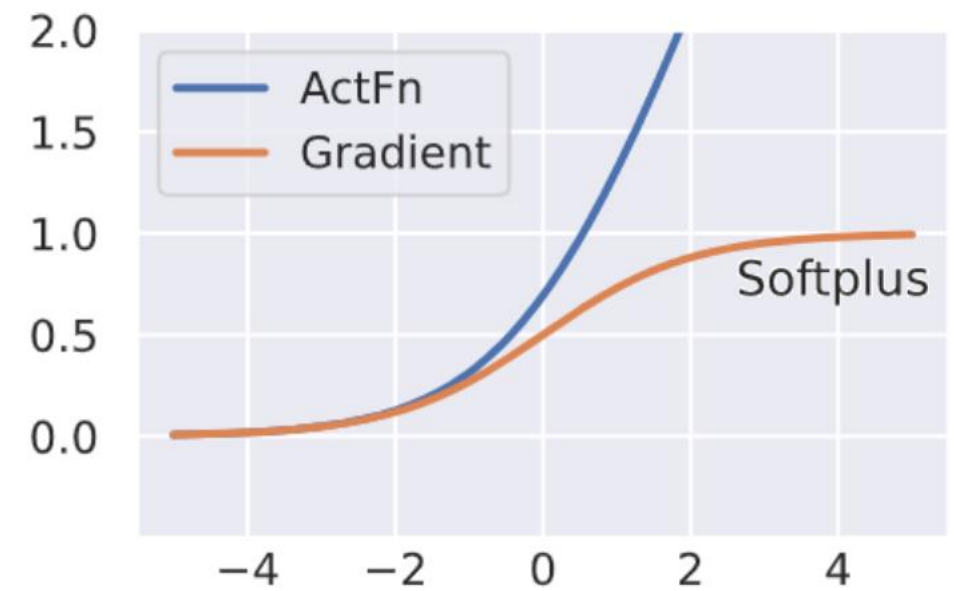
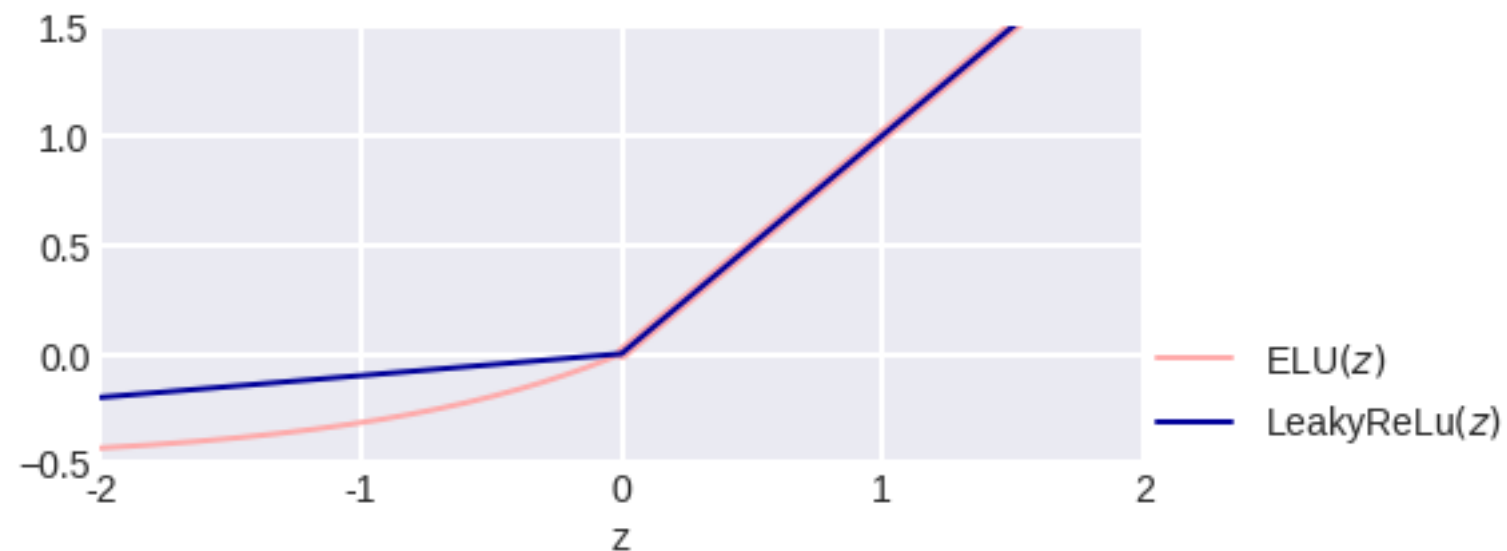
$$\text{softplus}(z) = \ln(1 + \exp(z))$$

Exponential Linear Unit

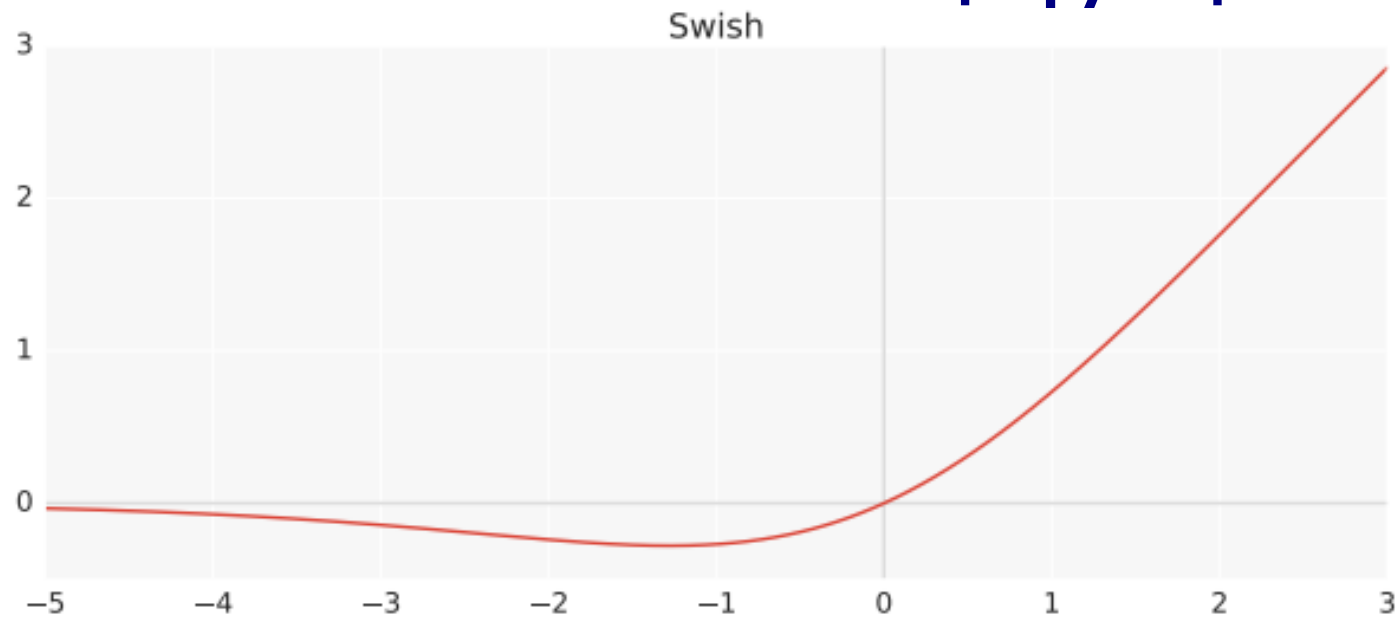
$$\text{ELU}(z) = \begin{cases} z, & z \geq 0, \\ \alpha(e^z - 1), & z < 0. \end{cases}$$

Scaled Exponential Linear Unit

$$\text{SELU}(z) = \lambda \text{ELU}(z)$$



Ещё функции активации



Swish

$$\text{swish}(z) = x \cdot \sigma(\alpha x)$$

При замене ReLu на swish в глубоких сетях немного (<1%) улучшается качество

SiLU

```
CLASS torch.nn.SiLU(inplace=False)
```

[\[SOURCE\]](#)

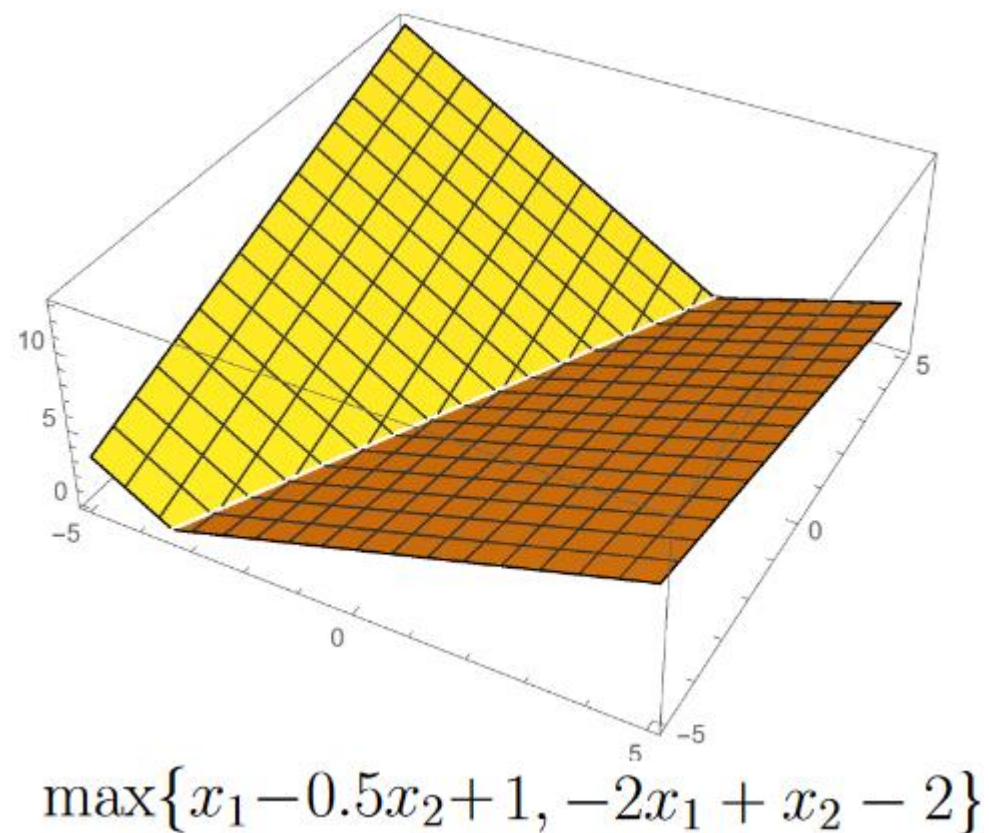
Applies the Sigmoid Linear Unit (SiLU) function, element-wise. The SiLU function is also known as the swish function.

$$\text{silu}(x) = x * \sigma(x), \text{ where } \sigma(x) \text{ is the logistic sigmoid.}$$

• NOTE

See [Gaussian Error Linear Units \(GELUs\)](#) where the SiLU (Sigmoid Linear Unit) was originally coined, and see [Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning](#) and [Swish: a Self-Gated Activation Function](#) where the SiLU was experimented with later.

Не совсем функции активации



Maxout

$$\text{Maxout}(z) = \max(w^T z + w_0, v^T z + v_0)$$

не совсем, т.к. есть параметры

GLU: Gated Linear Unit

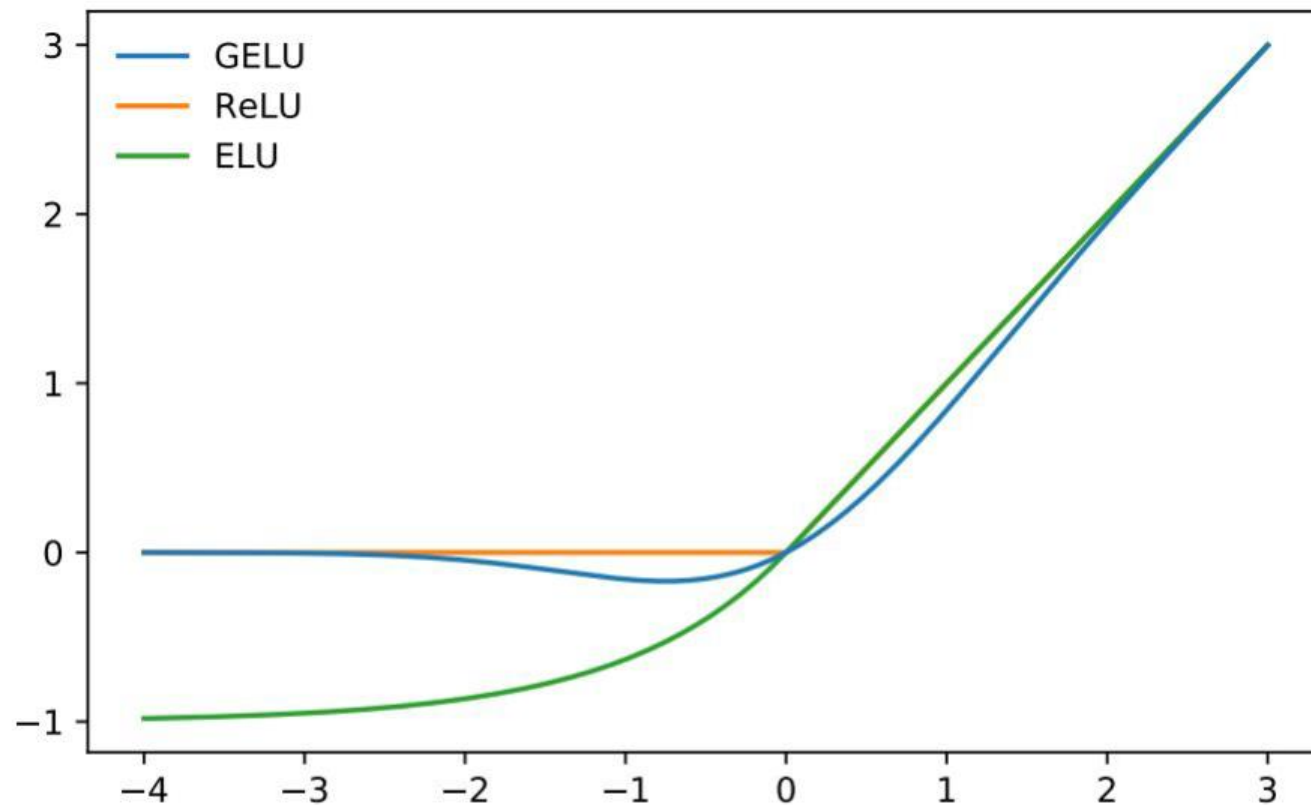
$$\text{GLU}(z) = \sigma(w^T z + w_0) \cdot (v^T z + v_0)$$

$$\text{GLU}(z) = \sigma(W^T z + w_0) \cdot (V^T z + v_0)$$

https://raw.githubusercontent.com/epfml/ML_course/master/lectures/08/lecture08d_neural_nets.pdf

Ещё функции активации

Gaussian Error Linear Unit
(Google's BERT, OpenAI's GPT-2)

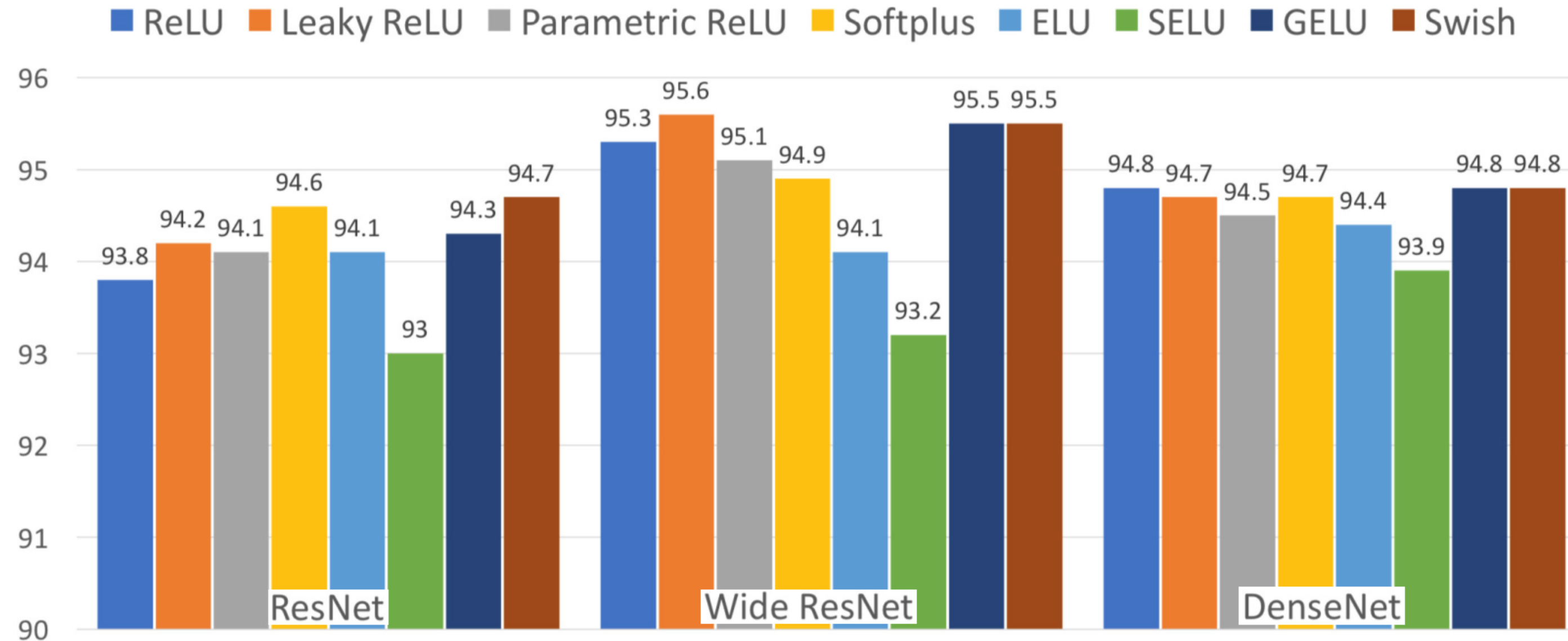


$$\text{GELU}(z) = \frac{z}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (z + \alpha z^3) \right) \right)$$

$\alpha = 0.044715$

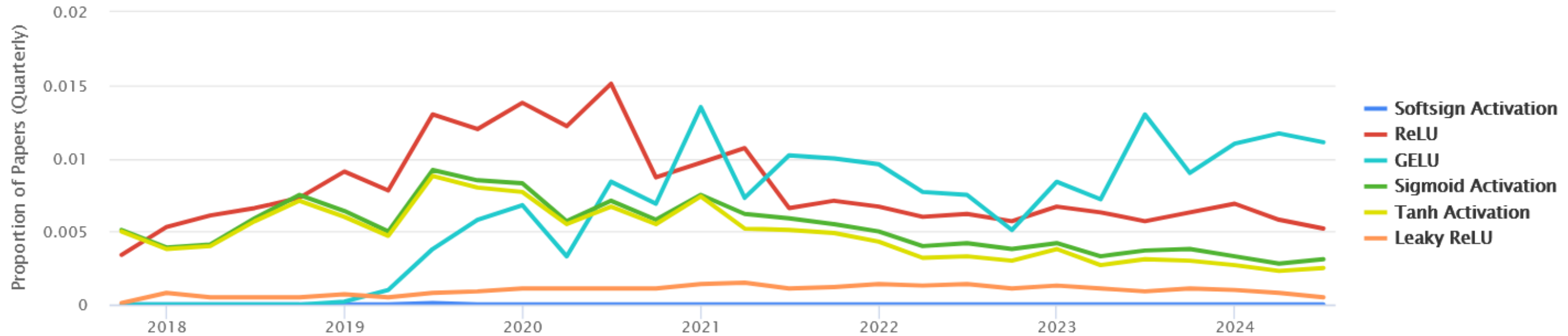
Figure 1: The GELU ($\mu = 0, \sigma = 1$), ReLU, and ELU ($\alpha = 1$).

Сравнение на CIFAR10



[Juhstin Johnson] / <https://arxiv.org/pdf/1710.05941.pdf>

«Популярность» функций активации



<https://paperswithcode.com/method/softsign-activation>

Сейчас этот сайт не активен:(

Минутка кода

```
# рекомендация для своей функции активации
@torch.jit.script
def fused_gelu(x):
    return x * 0.5 * (1.0 + torch.erf(x / 1.41421))

# своя активация в виде модуля
class ActivationFunction(nn.Module):

    def __init__(self):
        super().__init__()
        self.name = self.__class__.__name__
        self.config = {"name": self.name}

class MySigmoid(ActivationFunction):

    def forward(self, x):
        return 1 / (1 + torch.exp(-x))
```

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/tutorial3/Activation_Functions.html

Минутка кода

```
class LeakyReLU(ActivationFunction):

    def __init__(self, alpha=0.1):
        super().__init__()
        self.config["alpha"] = alpha

    def forward(self, x):
        return torch.where(x > 0, x, self.config["alpha"] * x)

def get_grads(act_fn, x):
    # Вычисляем производные
    x = x.clone().requires_grad_() # для этого тензора берём производные
    out = act_fn(x)
    out.sum().backward() # sum()!!!
    return x.grad # градиент

x = torch.tensor([-1., 0., 1.])
f = MySigmoid()
f(x), get_grads(f, x)

(tensor([0.2689, 0.5000, 0.7311]), tensor([0.1966, 0.2500, 0.1966]))
```

Проблемы НС

Большая сложность модели → переобучение
следующая лекция

Нестабильность обучения
по всему курсу

Большое время обучение, много данных
GPU, TPU и т.д.
сбор данных, аугментации и т.п.
эффективные современные алгоритмы настройки

Итог

НС – нелинейное обобщение линейных алгоритмов

- последовательное преобразование признакового пространства
 - ансамбль алгоритмов
- суперпозиция «логистических регрессий»
 - граф вычислений

высокая функциональная выразимость

обучение градиентными методами

Дальше: как делать эффективное обучение?

Попробуйте поиграться здесь:

<http://playground.tensorflow.org/>

Литература

Производные на компьютере

Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, Jeffrey Mark Siskind
«Automatic differentiation in machine learning: a survey» 2015-2018
<https://arxiv.org/abs/1502.05767>

лучшая «классическая» книга по DL

Ian Goodfellow, Yoshua Bengio, Aaron Courville «Deep Learning» <http://www.deeplearningbook.org/>

лучшая новая книга по DL

Simon J.D. Prince «Understanding Deep Learning»
<https://udlbook.github.io/udlbook/>

Большой обзор по функциям активации

Jagtap A. D., Karniadakis G. E. How important are activation functions in regression and classification? A survey, performance comparison, and future directions //Journal of Machine Learning for Modeling and Computing. – 2023. – Т. 4. – №. 1. // <https://arxiv.org/pdf/2209.02681.pdf>

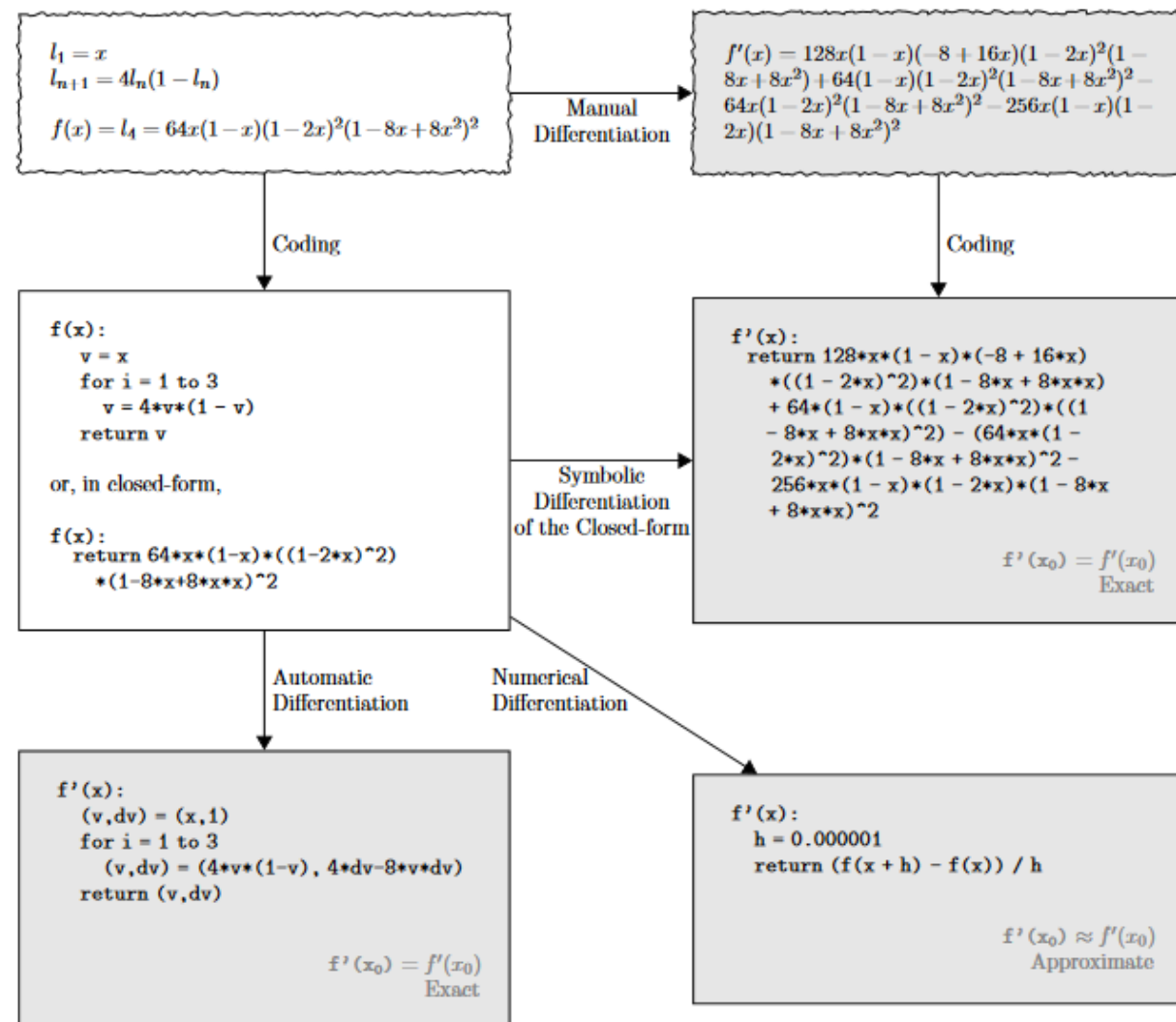


Figure 2: The range of approaches for differentiating mathematical expressions and computer code, looking at the example of a truncated logistic map (upper left). Symbolic differentiation (center right) gives exact results but requires closed-form input and suffers from expression swell; numerical differentiation (lower right) has problems of accuracy due to round-off and truncation errors; automatic differentiation (lower left) is as accurate as symbolic differentiation with only a constant factor of overhead and support for control flow.

Производные на компьютере:

Аналитическая формула
Численный градиент
Символьное дифференцирование
Алгоритмическое дифференцирование

